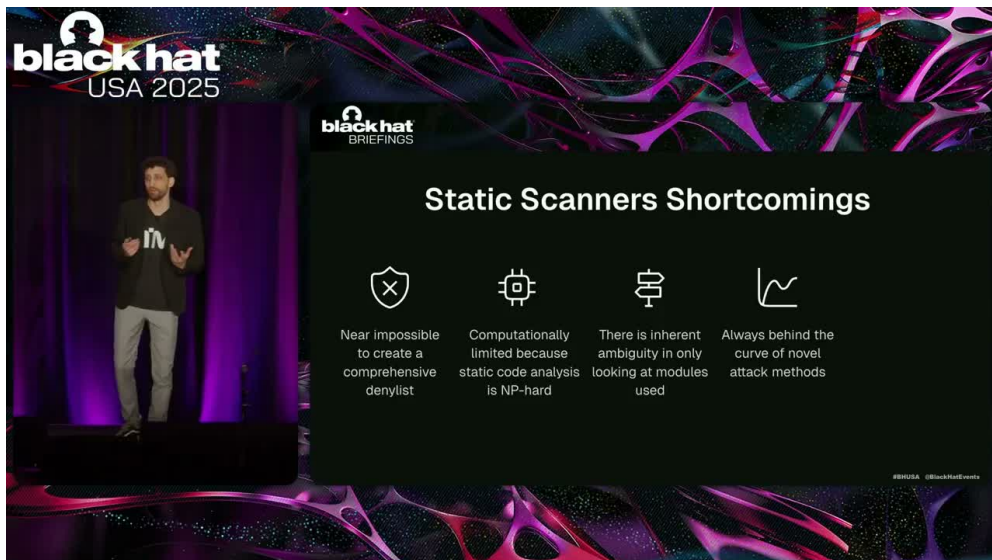


课程笔记

Black Hat USA 2025 | Advanced Bypass Techniques and a Novel Detection Approach

Black Hat / Itay Ravia & Codex

2026 年 5 月 16 日



视频作者/频道: Black Hat

发布日期: 2026-03-10

视频时长: 33:24

视频链接: <https://www.youtube.com/watch?v=jjiE9XzJo0M>

目录

1 开场：为什么模型扫描器会成为 AI 供应链安全入口	3
1.1 本章小结	4
2 模型文件的危险结构：权重、架构与不安全序列化	4
2.1 第三方模型的两类风险	4
2.2 权重与架构的分工	5
2.3 为什么架构需要被序列化	6
2.4 Pickle、Keras Lambda 与 SafeTensors 的边界	7
2.5 本章小结	8
3 静态扫描器的工作方式与第一层失效：告警、未知导入与人工判断	8
3.1 为什么先讨论静态扫描器	8
3.2 拒绝列表：已知危险名称的匹配	9
3.3 三个扫描器裁决：危险、安全与未知	10
3.4 本章小结	11
4 绕过一：拒绝列表无法穷尽 Python 生态	12
4.1 攻击面为什么无法枚举	12
4.2 MLflow local-run 风格包装入口	12
4.3 为什么这对维护者也不公平	14
4.4 本章小结	15
5 绕过二：嵌入字节码让静态分析撞上复杂性墙	15
5.1 从导入名走向程序行为	15
5.2 关于停机问题和 NP-hard 表述的谨慎理解	15
5.3 八行函数：拆字符串再导入 <code>os</code>	16
5.4 导入列表歧义：合法与恶意看起来一样	18
5.5 本章小结	18
6 绕过三：Pickle 虚拟机不同步攻击	18
6.1 Pickle 是一台小型栈机	19
6.2 扫描器的抽象解释会丢状态	20
6.3 从 <code>builtins.str.system</code> 到 <code>os.system</code> 的分叉	21
6.4 为什么这类错误会系统性出现	23
6.5 本章小结	24
7 绕过四：Joblib 的动态字节块让扫描提前失明	24
7.1 Joblib 是 Pickle 加动态 NumPy 字节块	25
7.2 扫描器停止，真实加载继续	26
7.3 格式复杂度带来的安全盲区	26
7.4 本章小结	27

8	为什么允许列表也卡住：YOLO、getattr、SafeTensors 与自定义架构	27
8.1	YOLO 与 Ultralytics：流行模型会逼迫清单扩张	29
8.2	getattr：一个合法又危险的模糊原语	30
8.3	SafeTensors 的 caveat：安全权重格式不等于安全模型仓库	30
8.4	auto_map、DeepSeek、Kimi K2：自定义架构如何进入加载路径	31
8.5	允许列表真正缺的，是上下文语义	32
8.6	本章小结	33
9	动态沙箱检测：从静态签名转向行为追踪	33
9.1	从早期杀毒签名到沙箱行为分析	33
9.2	正常模型行为边界：小得多，因而可定义	34
9.3	动态扫描器的形式化定义	35
9.4	四个动态结果：同一批绕过在行为层暴露	36
9.5	Keras Lambda：静态只能说可疑，动态能看到 system	38
9.6	为什么动态方案正好回应静态短板	39
9.7	静态与动态的反馈闭环	40
9.8	本章小结	40
10	总结与问答：动态检测的边界、优势与实践落点	40
10.1	演讲收束：为什么模型扫描仍然绕不开	41
10.2	问答一：截图里的工具到底在看什么	42
10.3	问答二：动态检测会不会变成无限枚举 RCE	43
10.4	实践落点：把动态扫描放进模型准入流程	43
10.5	仍然开放的问题与下一步	44
10.6	本章小结	44

1 开场：为什么模型扫描器会成为 AI 供应链安全入口

Black Hat USA 2025 这场分享的题目是“Smashing Model Scanners: Advanced Bypass Techniques and a Novel Detection Approach”。主持人在 00:00:05–00:00:13 介绍主题时，已经把问题压缩成两个关键词：模型扫描器和绕过技术。真正的教学主线从 00:00:38 左右开始，讲者 Itay Ravia 介绍自己是 Aim Security 研究部门 Aim Labs 的负责人，长期工作在网络安全与 AI 研究交界处。他把保护 AI 转型形容成一种不断处理新风险的过程：缓解了一个风险，另一个风险又会出现。



图 1: 开场标题页给出本讲的主线：模型扫描器会被系统性绕过，随后需要一种新的检测方法。这里的“model scanner”可以理解为 AI 供应链门口的安检机：它接收外部模型文件 M ，尝试在加载前判断风险 $V(M)$ 。标题中的“Smashing”直接提示后文会从攻击者视角拆解这个安检机的薄弱环节。

1

这段内容的作用超出会议开场白。它先把读者带到一个更大的背景：AI 供应链已经不再只包含代码库、容器镜像、依赖包和数据集，模型文件本身也成了供应链制品。一个团队从公开仓库下载模型，然后在笔记本、训练集群或内部推理环境中加载它，这一步看起来像是普通工程流程，实际已经把外部提供者交付的对象带进了本地执行边界。

第三方模型的定义

第三方模型指来自组织外部来源的模型，例如从 Hugging Face 或其他公开仓库下载的模型。它的权重、架构文件、配置文件、仓库代码和加载逻辑可能由外部作者控制。与自研模型相比，第三方模型带来的核心问题是信任边界外移：数据科学家拿到的是一个可被加载和运行的制品，而不只是论文里的方法描述。

讲者在 00:01:38–00:02:02 给出整场路线图：先讨论使用第三方模型有什么风险；再讨论现有保护措施为什么存在内在缺陷；最后展示一种新的检测方法，尝试缓解这些缺陷。这个顺序很关键，因

¹源视频时间：00:00:45；来源区间：00:00:45–00:00:46。

为模型扫描器的地位来自前两步：第三方模型可能带来执行风险，组织又需要某种入口检查，于是扫描器成了模型进入研发流程前的安检门。

更尖锐地说，本讲真正追问的是下面这个问题：当一个未知模型文件来自公开仓库时，扫描器能否判断加载它或推理它时会不会执行攻击者控制的代码？如果答案经常落在“未知”“看起来安全”“需要人工判断”这些灰区，模型扫描器就会从安全入口变成风险传送带。

本讲的核心安全问题

模型扫描器要守住的第一道门，是模型加载或推理期间的代码执行风险。后门输入、输出操纵、模型幻觉和对输出的过度信任都值得讨论，但本讲优先处理更直接的主机风险：模型文件在加载阶段或推理阶段触发恶意代码，进而影响笔记本、服务器、CI 环境或内部网络。

把模型扫描器理解成 AI 供应链入口，有一个朴素类比：传统软件供应链会扫描依赖包中的恶意脚本、安装钩子和危险调用；AI 供应链则需要扫描模型制品中的加载逻辑、序列化结构和框架特性。差别在于，模型文件经常披着“数学对象”的外衣，容易让人误以为它只是一堆数字。讲者接下来要拆掉的，正是这种误解。

1.1 本章小结

开场部分完成了问题定位：第三方模型已经是 AI 供应链制品，模型扫描器承担的是准入闸门角色。本讲优先追问加载和推理期间的代码执行风险，因为它会把“模型是否可信”的问题推到“宿主环境是否会被接管”的层面。下一章开始拆模型文件本身，说明为什么权重、架构和序列化格式会把安全问题带进模型文件内部。

2 模型文件的危险结构：权重、架构与不安全序列化

2.1 第三方模型的两类风险

从 00:02:04 开始，讲者把第三方模型风险分成两层。第一层是代码执行：下载并加载来自 Hugging Face 或其他来源的模型时，恶意代码可能运行在本机或组织环境中。讲者明确强调，这种执行可以发生在加载时，也可以发生在推理时。加载时执行类似“打开包裹时触发机关”；推理时执行类似“只有收到特定输入才启动隐藏路径”。二者都属于模型制品越过信任边界后的主机侧风险。

第二层是模型行为风险：训练者或发布者控制模型输出，因此模型可能包含后门输入、输出操纵或不可信响应。00:02:30–00:02:55 的重点是提醒读者，第三方模型的输出也要按不可信内容处理。可是讲者随后把焦点收回到代码执行，因为它的严重性更高：一旦模型加载代码能运行命令，攻击面就从“模型回答是否可靠”升级为“运行环境是否已经被接管”。

不要把模型风险压扁成“输出不可信”

输出不可信主要影响下游决策，代码执行直接影响宿主环境。前者像收到一份可能误导人的报告，后者像把陌生人带进机房并给了他执行命令的机会。本讲后续所有绕过案例，都围绕加载或推理期间的代码执行展开。

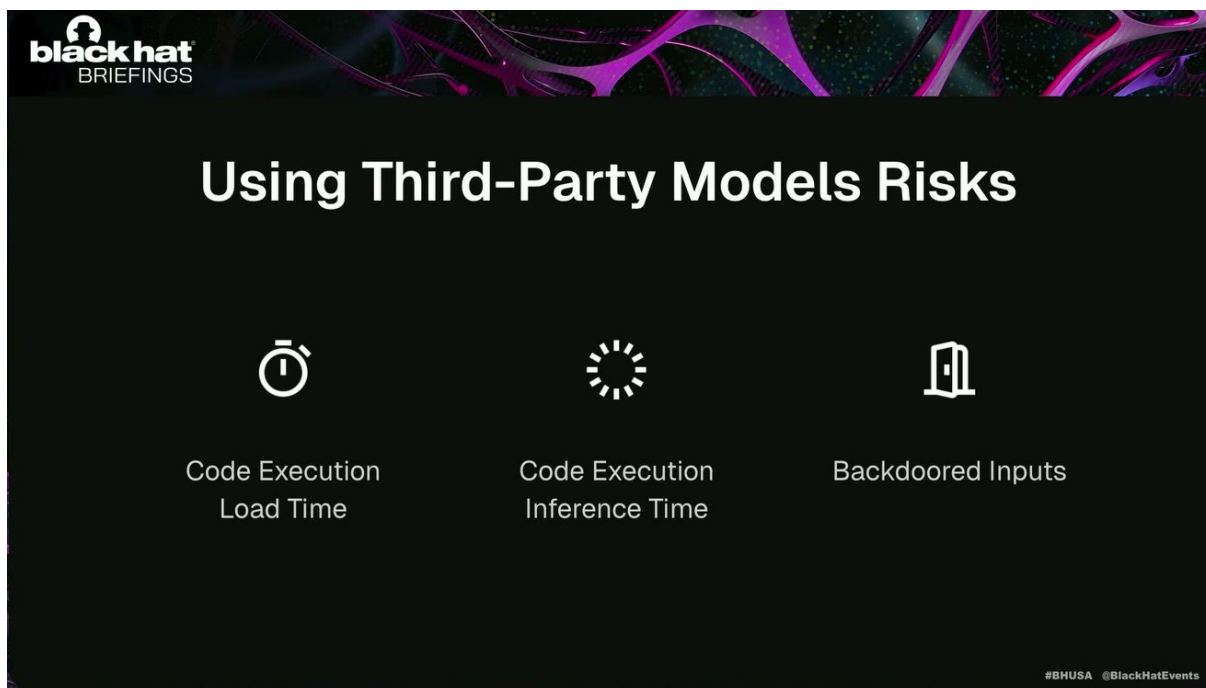


图 2: 第三方模型的三类风险被放在同一页: 加载时执行代码、推理时执行代码、后门输入。前两者是更直接的主机环境风险, 因为模型文件一旦把反序列化、加载逻辑或推理钩子变成执行入口, 数据科学家的本地机器或训练环境就会成为攻击面。可以把风险写成 $B(M) = 1$: 恶意模型 M 通过检查后仍触发异常行为。²

2.2 权重与架构的分工

00:03:04–00:04:05 是理解整场演讲的地基。讲者先定义模型权重: 权重是训练后得到的数值参数, 存放模型学到的统计信息。现代模型可能有数百万、数十亿, 甚至更多参数。单独看权重, 它们像一仓库电子元件: 每个元件有阻值、电容或芯片状态, 但元件堆在一起并不会自动形成收音机、显卡或雷达。

架构则是另一件东西。模型架构定义权重之间的连接、计算顺序和组合关系, 讲者用一个直观例子说明: 架构可能规定第 1,000,000 个权重要与第 27 个权重相加, 再乘以第 512 个权重, 这只是模型内部的一条小电路。更大的模型包含海量类似关系, 因此架构像电路图、装配说明和运行规则的集合。

用一个简化公式可以把这个拆分写清楚:

$$M = (W, A)$$

- M : 待加载、待检查的模型制品。
- W : 模型权重, 也就是训练得到的数值参数张量。
- A : 模型架构和加载逻辑, 包括计算图、类定义、框架配置、字节码或仓库中的自定义代码。

这条公式的普通话解释很简单: 一个可运行模型由“数值内容”和“如何使用这些数值的结构”共同构成。只检查 W , 就像只检查零件材质; 安全风险常常藏在 A , 也就是连接方式、加载方式和

²源视频时间: 00:02:40; 来源区间: 00:02:40–00:02:41。

执行路径里。

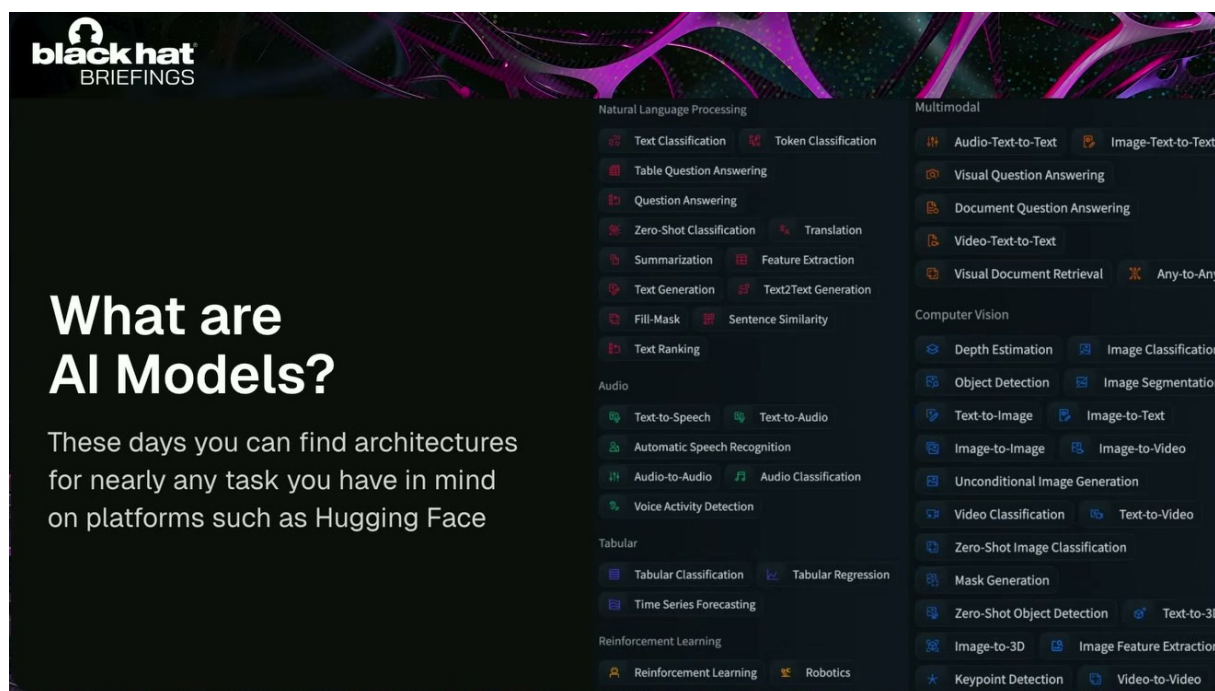


图 3: 讲者用 Hugging Face 任务版图说明今日模型仓库的规模与复杂度: 文本、音频、视觉、多模态任务各自牵出不同架构。模型因此不只是权重矩阵 W , 还包含结构与加载逻辑 A , 更像 $M = (W, A)$ 。权重像器件参数, 架构像电路连线图; 只有两者一起出现, 框架才知道怎样把张量计算起来。³

模型文件并不等同于权重文件

权重回答“模型学到了哪些数值参数”, 架构回答“这些参数怎样被组合和执行”。安全扫描若只把模型当成数字矩阵, 就会漏掉架构、加载器和自定义函数里携带的执行能力。

2.3 为什么架构需要被序列化

00:04:07–00:04:52 解释了为什么这个问题会在现实研发中高频出现。数据科学家接到一个任务, 例如问答、时间序列预测或视频生成, 通常能在网上找到相近模型。讲者提到, Hugging Face 这类仓库已经承载接近两百万个模型; 数据科学家每天下载、试验、微调这些模型。很多时候, 他们会调整权重或继续训练, 但复用原有架构。

这里引出“序列化”的必要性。序列化是把内存里的对象、模型状态或架构描述编码成字节, 以便存储、传输和再次加载。反序列化则是反向过程: 框架读取字节, 把模型对象重建出来。对简单数值数组来说, 这个过程可以很克制; 对复杂架构来说, 序列化经常要保存类名、函数、配置、计算图、字节码, 甚至加载时需要执行的构造逻辑。

序列化与不安全反序列化

序列化像把一台机器拆成箱子并写上装配说明; 反序列化像按说明重新装起来。风险来自“不安全反序列化”: 加载器在重建对象时允许输入数据指定要构造的类、要调用的函数或要执行的代码。攻击者一旦控制这些字节, 模型文件就可能变成执行载体。

³源视频时间: 00:04:07; 来源区间: 00:04:07–00:04:08。

这也解释了为什么模型扫描器被推到入口位置。对很多组织来说，下载模型已经成为日常研发动作：反复下载、比较、微调和部署模型。每一次加载第三方模型，都是一次供应链准入决策。扫描器如果无法理解序列化结构，就难以回答“这个模型加载时会做什么”。

2.4 Pickle、Keras Lambda 与 SafeTensors 的边界

00:04:55–00:06:24，讲者列举了机器学习框架和格式的现实复杂度。他略过完整列表，抓住三个代表：Pickle、TensorFlow/Keras 的 Lambda 机制，以及 SafeTensors 的误解。

Pickle 是 Python 的对象序列化格式，优势是表达能力强：它能保存并重建复杂 Python 对象。问题也在同一个地方：Pickle 在反序列化时可以导入模块、构造对象、调用函数。对可信本地对象来说，这很方便；对不可信模型文件来说，这等于把“如何重建对象”的权力交给了文件作者。许多框架仍然在序列化链路中使用 Pickle，原因并不神秘：复杂架构需要被表示，而 Pickle 正好能表示很多 Python 世界里的复杂对象。

Pickle 的危险来自表达能力

Pickle 危险的根源不在某个单独函数名，核心在于它的反序列化过程具备对象构造和函数调用能力。它像一套小型指令语言，能够描述“取这个模块、拿这个函数、用这些参数调用”。扫描器后续遇到的很多困难，都来自这种指令级表达能力。

TensorFlow/Keras 展示了另一个层次的问题。即便底层文件使用 JSON 或 Protobuf 这类相对克制的格式，周围的加载逻辑仍可能引入危险。讲者在 00:05:32–00:05:58 指出，Keras 可以把 Python lambda 函数嵌入模型文件。Keras Lambda 的正常用途是表达自定义层或小段自定义计算；风险在于，这段 Python 函数也可能包含恶意行为。换句话说，安全问题不只发生在 Pickle 这种“坏名声”格式里，也会发生在看起来更结构化的框架功能周边。

SafeTensors 则代表一种常见误解。它的设计目标是安全地存储张量，避免 Pickle 式任意对象加载；在“只存张量”这个问题上，它确实更克制。可讲者在 00:06:00–00:06:24 提醒，实际加载流程里仍可能出现从 SafeTensors 模型走向 Pickle 加载的路径，后续还会讨论 SafeTensors 与自定义架构相关的其他缺陷。这个 caveat 的要点是：SafeTensors 能降低权重存储层的风险，但它无法替代对架构代码、加载逻辑和仓库附带 Python 文件的审查。



ML Frameworks & Formats

ML Framework	Model file formats	Serialization format
PyTorch	PyTorch ZIP PyTorch legacy	Pickle inside Zip Pickle
Tensorflow	Keras v3 Keras legacy SavedModel	"Json" HDF5 "Protobuf"
Transformers	SafeTensors ...	"Json" + SafeTensors
MLflow	-	Pickle Cloudpickle (still pickle...)
Joblib	Joblib	Joblib pickle
ONNX	ONNX	Protobuf

#BHUSA @BlackHatEvents

图 4: 框架与格式把风险落到具体载体: PyTorch、MLflow、Joblib 都会碰到 Pickle 或 Pickle 变体; TensorFlow/Keras 走 JSON、HDF5、Protobuf 一类容器时仍可能承载自定义逻辑; SafeTensors 只约束张量存储, 后续自定义架构代码仍可能参与加载链。序列化是把对象折叠成字节, 反序列化是把字节重新展开; 一旦展开过程允许调用函数, 文件就带上了行为。⁴

本节压缩结论

模型制品的危险结构来自 $M = (W, A)$ 中的 A 。权重 W 多数情况下是数值张量; 架构 A 为了表达复杂计算, 常常依赖代码、字节码或框架特定序列化。Pickle、Keras Lambda 和 SafeTensors caveat 共同说明: 模型安全不能只问“权重格式叫什么”, 还要问“加载时哪些逻辑会被执行”。

2.5 本章小结

到 00:06:28, 讲者完成了从“为什么需要扫描模型”到“为什么模型文件值得被扫描”的铺垫。第三方模型是外部供应链制品; 核心风险是加载或推理期间的代码执行; 后门输入和输出操纵存在, 但在本讲中属于次级风险。模型可以被概念化为 $M = (W, A)$: W 是数值权重, A 是架构与加载逻辑。真正麻烦的地方在于, A 往往必须通过序列化、框架配置、字节码或自定义代码来表达。于是, 模型扫描器要面对的是一个可能携带执行语义的工程包, 静态数字表这个比喻已经失真。

3 静态扫描器的工作方式与第一层失效: 告警、未知导入与人工判断

3.1 为什么先讨论静态扫描器

前两章已经把问题推到台面上: 模型文件可能携带可执行逻辑, 加载模型或做一次推理时, 攻击者就可能获得本地环境里的代码执行能力。于是自然会出现一个安全闸门: 把下载来的模型先交

⁴源视频时间: 00:05:06; 来源区间: 00:05:06–00:05:07。

给扫描器，让扫描器告诉数据科学家这个模型能不能用。

演讲者把今天讨论的现有方案称为**静态扫描器**。所谓“静态”，指它读取模型文件的字节、导入、操作码、序列化结构或框架算子，却不真正执行模型加载和推理。它像机场安检里的 X 光机：看包里有什么形状、什么密度、什么组合，但不把包里的机器打开运行。这个设计有明显好处，速度快、风险低、适合放进模型仓库或下载流程；代价也同样硬，扫描器看到的是“表示”，真正危险发生在“执行”。

可以把静态扫描器写成一个很小的抽象：

$$S_{\text{static}}(M) \rightarrow V(M)$$

其中：

- M ：被检查的模型制品，可能包含权重、序列化架构、嵌入函数或仓库代码线索。
- S_{static} ：不运行 M 的扫描过程，它只分析模型字节和可提取的结构。
- $V(M)$ ：扫描器给出的裁决，典型取值可以写成 $\{\text{safe}, \text{unknown}, \text{unsafe}\}$ ，分别对应“看起来安全”“无法判断”“已发现危险证据”。

静态扫描器的核心限制

静态扫描器判断的是“文件里能看见什么”，实际攻击利用的是“加载或推理时会做什么”。二者有关联，却不相同。只看导入列表、函数名或操作码，很容易把真正的行为压缩成一个不完整的影子。

3.2 拒绝列表：已知危险名称的匹配

最常见的静态策略是拒绝列表。拒绝列表 L_{deny} 记录一组已知危险的模块、函数或操作，例如 `os.system`、`posix.system` 这类命令行执行入口。扫描器从模型里提取指标集合 $I(M)$ ，如果交集命中，就把模型判为危险：

$$I(M) \cap L_{\text{deny}} \neq \emptyset \Rightarrow V(M) = \text{unsafe}$$

其中：

- $I(M)$ ：从模型制品里抽取出的导入、函数、模块、操作码或框架算子。
- L_{deny} ：安全团队维护的已知危险指标集合。
- `unsafe`：扫描器认为模型包含不应出现在模型加载路径里的危险能力。

以 Pickle 为例，如果反序列化流里出现 `posix.system`，在 Linux 上它实际对应 `os.system` 一类命令执行能力。一个普通模型没有理由在加载权重时运行任意 CLI 命令；所以 Hugging Face PickleScan 一类工具看到这个指标时，把它标成恶意或不安全，是合理的。

演讲者特别强调，风险不限于 Pickle。Keras 的 Lambda 函数、TensorFlow/Keras 的自定义逻辑、Dill 与 Cloudpickle 支持的函数序列化，都可能把可执行逻辑嵌入模型生态。换句话说，拒绝列表面对的对象不只是一种文件格式，还包括整个 Python 与机器学习框架的行为表达系统。

拒绝列表与允许列表

拒绝列表像“通缉名单”：看到名单上的危险名称就拦截。允许列表像“准入名单”：只有名单内的对象可以通过。前者容易漏掉新包装、新别名和新库；后者容易误伤真实模型，因为机器学习生态大量依赖自定义架构、非标准库和动态机制。

3.3 三个扫描器裁决：危险、安全与未知

演讲者用一组三栏示例解释现实中的静态扫描结果。左侧是恶意 Pickle：扫描器明确识别 `posix.system`，给出不安全裁决。中间是只序列化权重的模型文件：扫描器认识其中导入，认为这些导入安全，于是放行。右侧最关键：哪怕只是把一个最简单的线性架构嵌进模型文件，扫描器也可能遇到未知导入，最后把问题推回给数据科学家。

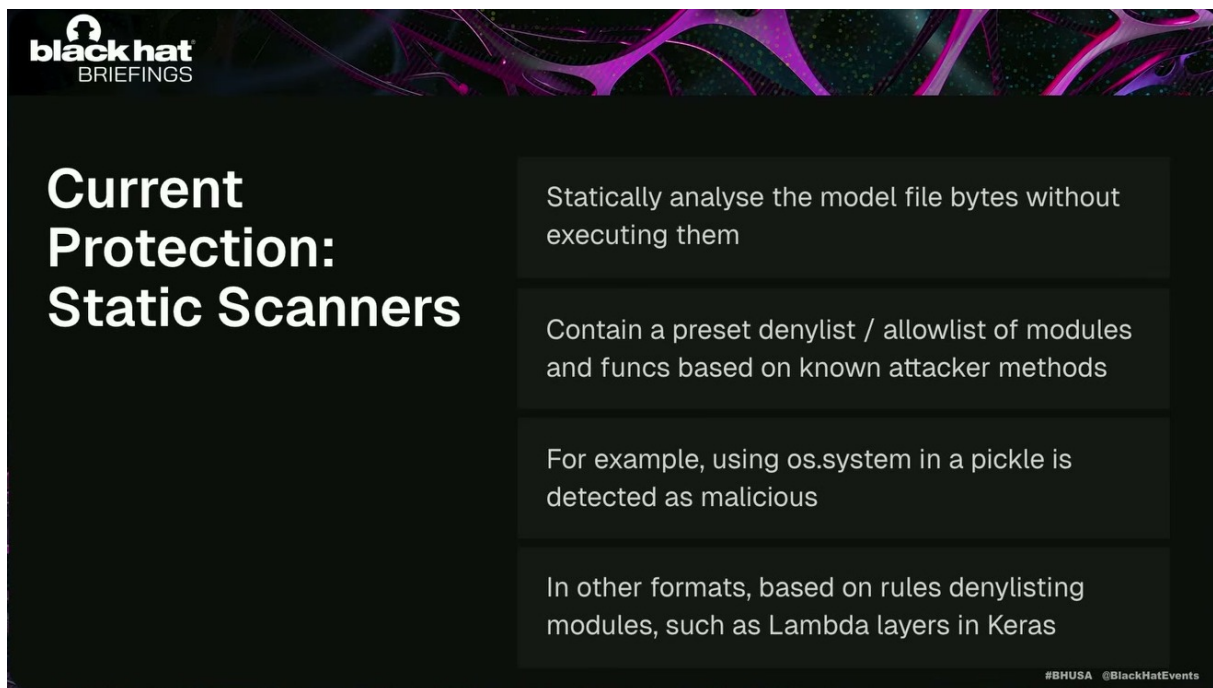


图 5：静态扫描器的工作方式被压缩成三句话：读取模型文件字节、依据已知危险模块和函数做拒绝列表或允许列表匹配、在 Pickle 中发现 `os.system` 一类入口时报警。形式化地说，静态扫描器 S_{static} 不执行模型，只从字节中抽取指标并输出 $V(M)$ 。它像只看包裹外观和 X 光片的安检，速度快，代价是无法观察真实运行行为。⁵

⁵源视频时间：00:06:45；来源区间：00:06:45–00:06:46。

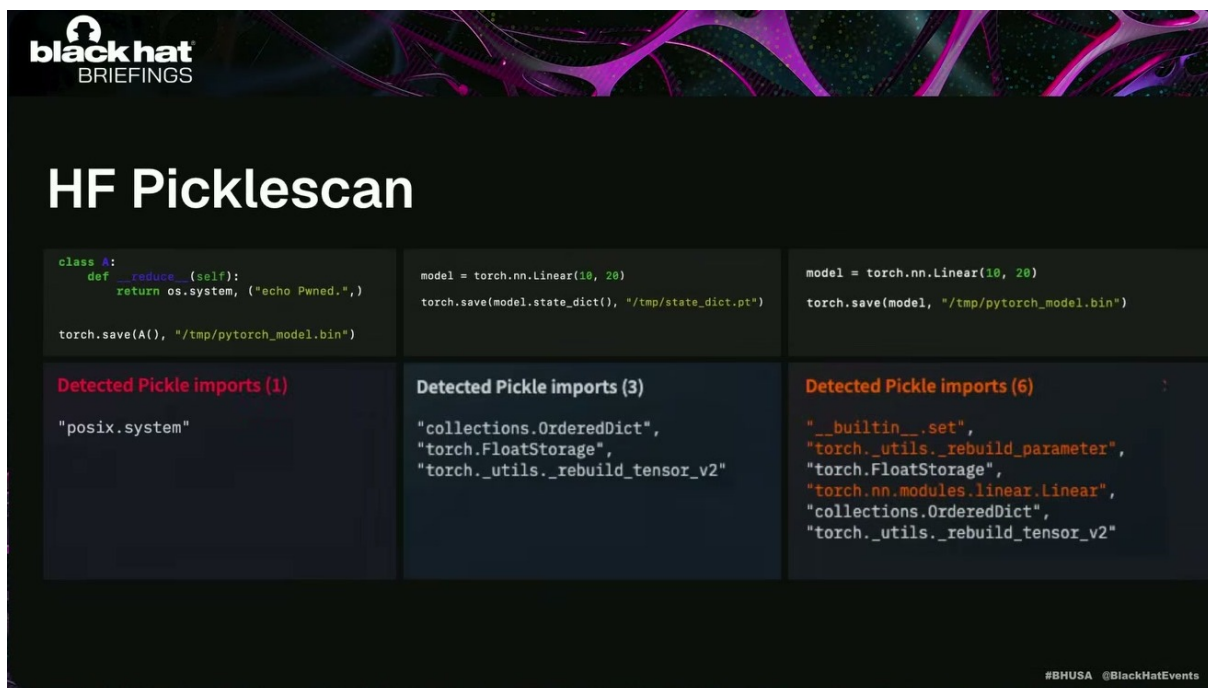


图 6: HF Picklescan 三联例子展示静态判断的第一层张力：左侧恶意 Pickle 因 `posix.system` 被抓住；中间权重字典只出现常见安全导入；右侧完整模型包含更多重建类与模块导入，开始进入“已知安全、已知危险、未知待判”的灰区。对使用者来说，未知导入如果被流程默认为放行，就会把人工判断变成供应链中的薄弱点。⁶

这个右侧案例才是日常工作里的麻烦源头。一个数据科学家要下载模型、跑实验、做微调，本来关注的是数据、损失函数、评估指标和训练成本；突然被要求判断某个未知导入是否安全，相当于让乘客自己解释 X 光机里的可疑阴影。多数人没有上下文，也没有时间审计每个库、每个导入、每个序列化结构。

“unknown”并非中性状态

在自动化流水线里，unknown 很容易变成事实上的放行。原因很朴素：业务要继续，实验要跑，模型要评估。只要扫描器没有明确阻断，未知导入就会把安全判断从工具链转移到人身上。

这正是第一层失效：静态扫描器能抓住一部分已知危险，也能识别一部分简单安全场景；一旦导入或操作落入灰区，它给出的不再是安全保证，只剩一张“请自行判断”的纸条。

3.4 本章小结

本章建立了后面绕过案例的基线：静态扫描器 S_{static} 不执行模型，只从字节、导入、函数名、操作码和框架算子里抽取线索，并输出 $V(M)$ 。拒绝列表能抓住 `os.system`、`posix.system` 这类已知危险入口，却无法自动理解未知导入的真实行为。最危险的地方在于，未知裁决把专业安全判断转嫁给数据科学家，也给攻击者留下了绕开“已知危险名称”的空间。

在进入四个具体绕过之前，需要先把讲者的范围限定说清楚。00:09:04–00:09:24 中，讲者强调他们实际找到了数十类绕过方法，而且每一类方法还可以派生出许多具体样本；演讲只展示四种，是

⁶源视频时间：00:07:22；来源区间：00:07:22–00:07:23。

因为时间有限，也因为这四种足以覆盖静态扫描器的几类结构性弱点。因此，下面的四章更适合被理解为代表性剖面，而非完整攻击清单：它们分别打在名单维护、字节码语义、Pickle 状态模拟、格式解析边界和允许列表生态压力上。

4 绕过一：拒绝列表无法穷尽 Python 生态

4.1 攻击面为什么无法枚举

第一种绕过利用的是拒绝列表的天然缺口。Python 生态拥有成千上万的包，每个包又有大量函数、类、入口点和内部包装层。包还在持续更新，今天没有的函数明天可能出现，今天只是工具函数的代码明天可能改成调用子进程。想靠一个静态名单列完所有危险入口，工程上会迅速撞到规模墙。

用集合语言说，拒绝列表试图维护 L_{deny} ，也就是所有危险函数名的集合。但真实生态里的危险路径更像一个图：许多看似无害的节点，最终都可能连到同一个危险汇点，例如命令行执行、进程创建、网络连接或文件写入。扫描器只盯着少数显眼节点，攻击者就可以从旁边的节点绕过去。

包装函数绕过

包装函数（wrapper function）本身的名字未必在拒绝列表里，但它内部会调用危险汇点。对扫描器来说，模型只导入了一个陌生函数；对真实运行环境来说，这个陌生函数最终仍会触发命令执行。

演讲者的验证方式很直接：他们让一个 AI Agent 搜索这类包装函数，约两个小时内找到 50 个围绕不安全函数的包装入口。这个数字不该被理解成“只有 50 个”，它说明绕过候选很容易被系统化搜索出来。安全团队维护名单的速度，很难压过包生态和自动化搜索的组合速度。

4.2 MLflow local-run 风格包装入口

演讲中的具体例子是 MLflow 的本地运行入口，类似 `mlflow.projects.backend.local.run` 这一类 local backend run 函数。MLflow 是机器学习工程里常见的实验、项目和模型管理工具；它的本地运行能力本来服务于实验复现与项目执行，底层会涉及命令行、进程和环境调用。攻击者把这类入口放进模型加载路径后，静态扫描器看到的是一个不在拒绝列表里的 MLflow 导入，真实执行时却可能在本地环境里跑 CLI 命令。

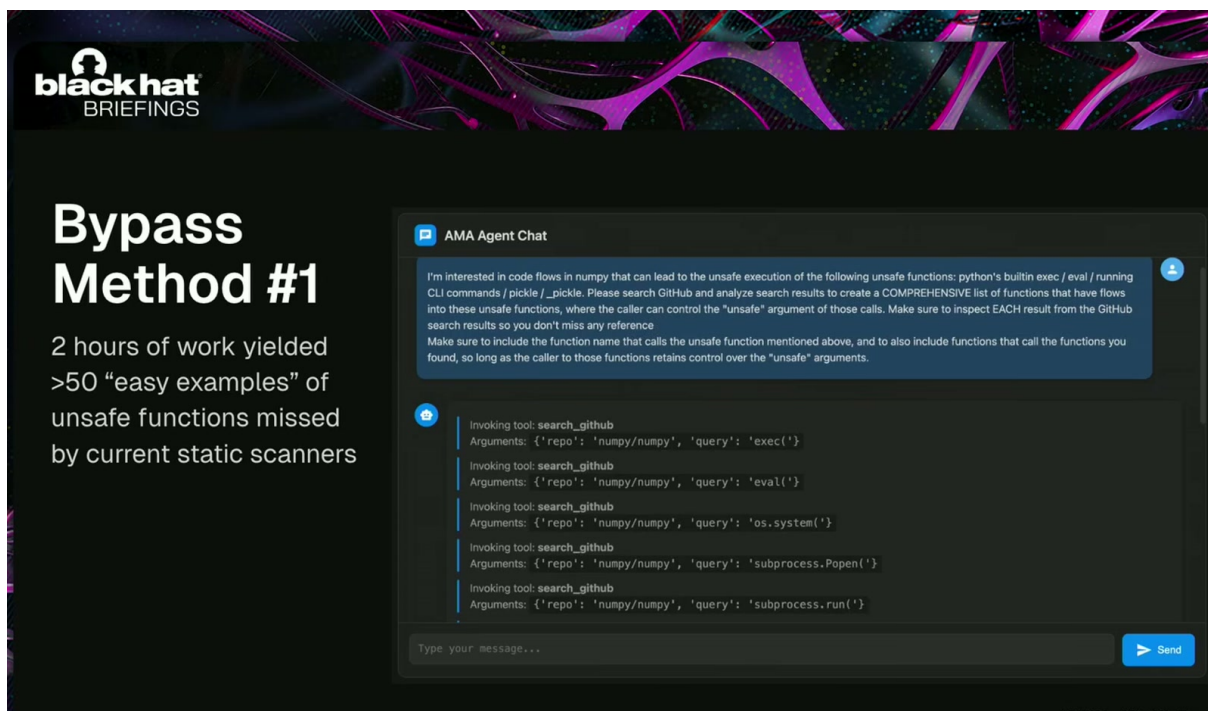


图 7: 拒绝列表的第一处硬伤: 攻击者不必直接写出 `os.system`, 只要找到会间接触发命令执行的包装函数, 就能绕开只认固定危险名称的静态扫描器。图中 AI agent 在搜索 Python 生态里的潜在包装入口。拒绝列表可以看成“黑名单集合” L_{deny} , 扫描器只检查 $f \in L_{\text{deny}}$, 问题在于危险行为可能藏在 $g \rightarrow f$ 的调用链里。⁷

⁷源视频时间: 00:10:34; 来源区间: 00:09:30–00:11:15。

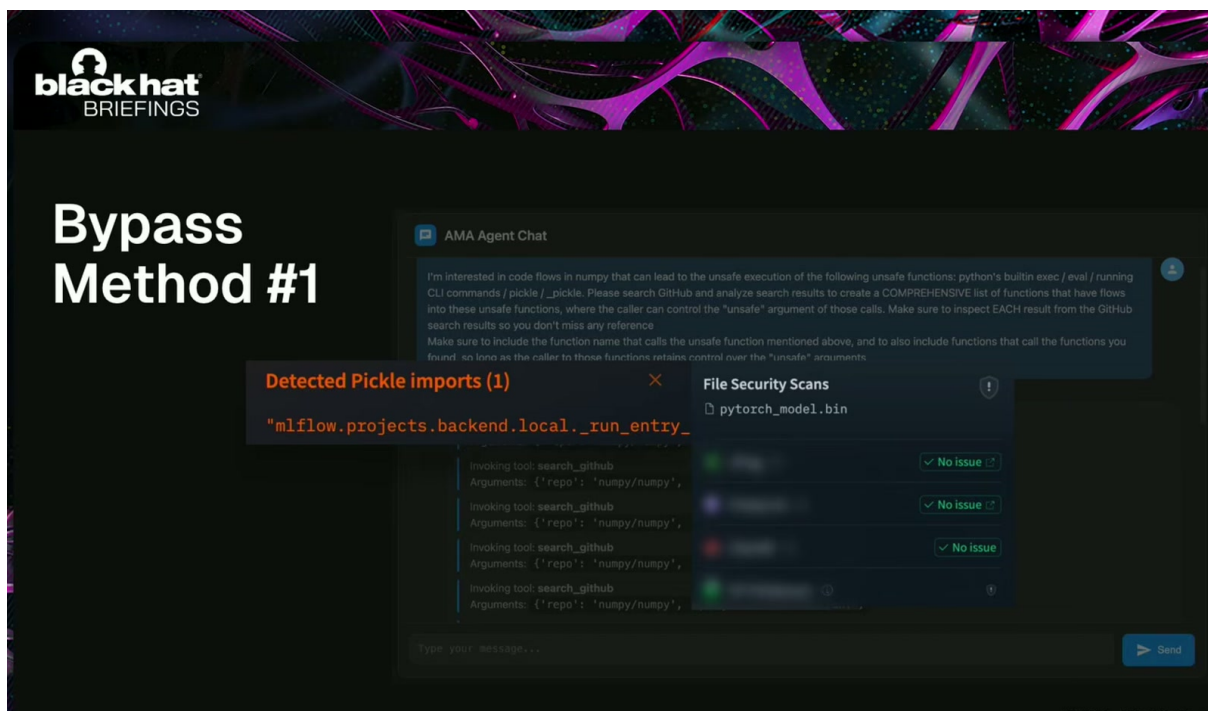


图 8: MLflow wrapper 示例: 扫描器看到的是 `mlflow.projects.backend.local._run_entry_` 一类导入, 而底层语义可以通向本地命令执行。右侧多个文件安全扫描结果显示“无问题”或未阻断, 这说明未知包装名很容易落入事实上的放行区。⁸

这类案例的欺骗性很强: `mlflow` 这个名字在机器学习语境中很正常, `projects.backend.local.run` 也不像 `os.system` 那样一眼刺眼。数据科学家看到未知导入后, 可能会去查扫描器建议的外部工具或供应商输出; 演讲者展示的结果是, 这些工具把它称为安全或没有明确拦截。于是攻击链的关键一步完成了: 危险能力从“已知坏名字”变成“陌生但像正常 ML 工具链的名字”。

名字安全感会误导判断

安全判断不能停在“这个库听起来正常”。正常库也可能提供进程执行、命令运行、插件加载、远程资源拉取等能力。攻击者喜欢借用真实生态的常用工具, 因为它们既有危险能力, 又有可信外观。

4.3 为什么这对维护者也不公平

演讲者没有把矛头指向某个具体扫描器作者。他甚至提到自己也写过静态扫描器。问题的残酷之处在于: 这并非把名单写得更快就能完全解决的维护缺陷。拒绝列表面对的是开放生态、动态依赖和间接调用。

可以把它类比成防诈骗短信。早期拦截“中奖”“转账”“验证码”这类关键词有效; 诈骗者随后会把关键词拆开、换同义词、换平台、换正常业务名。拒绝列表仍有价值, 它能拦住低成本、已知、重复的攻击; 它无法承诺覆盖所有表达方式。

⁸源视频时间: 00:11:14; 来源区间: 00:09:30–00:11:15。

危险汇点与中间包装

安全分析里常把真正产生副作用的位置叫作“汇点”(sink)，例如 shell 执行、网络发送、敏感文件读取。包装函数是通往汇点的中间道路。拦 `os.system` 只是拦住一条大路，生态里还存在小路、桥、隧道和后来新修的岔路。

4.4 本章小结

拒绝列表扫描器的第一类结构性弱点，是它只能编码已知危险名称。Python 生态里的包装函数可以把同一个危险汇点藏在陌生导入之后；MLflow local-run 风格入口展示了这种路径如何在机器学习语境中显得“正常”。AI Agent 两小时找到 50 个包装入口的实验说明，攻击者搜索绕过候选的成本很低，而扫描器维护完整拒绝列表的成本会持续上升。

5 绕过二：嵌入字节码让静态分析撞上复杂性墙

5.1 从导入名走向程序行为

第二种绕过把静态扫描器拖进更深的水里。Dill、Cloudpickle 以及 Keras Lambda 等机制允许把 Python 函数或字节码嵌入模型文件。字节码是源代码经过编译后的低层表示，仍然表达真实程序行为。它像一份拆成小指令的食谱：每一步看起来简单，组合起来可以动态拼出模块名、导入模块、取属性、调用函数。

这时扫描器面对的任务已经改变：它不再只是看“有没有导入 `os`”，还要判断一段程序在运行后会不会拼出 `os`、导入它、再调用 `system`。这个任务从名称匹配升级为程序语义分析。

嵌入字节码的关键变化

只看导入列表时，扫描器处理的是静态标签；分析嵌入字节码时，扫描器必须推断程序执行后的值、分支、调用和副作用。后者天然更难，很多情况下还会遇到理论层面的不可判定与工程层面的组合爆炸。

5.2 关于停机问题和 NP-hard 表述的谨慎理解

演讲者用“停机问题的泛化”和“NP-hard”来描述静态清洗这类函数的困难。这里需要说得精确一点：如果目标是对任意程序精确预测运行行为，那么会碰到停机问题一类不可判定障碍。停机问题问的是：给定任意程序 P 和输入 x ，能否总是判断 $P(x)$ 会停止。经典结论是，一般情形下不存在这样一个万能判定器。

NP-hard 则来自另一层直觉：即使把问题限制在某些可判定子集，路径条件、变量取值、字符串拼接、动态导入、反射调用等组合也可能让分析成本爆炸。严格说，停机问题和 NP-hard 属于不同理论范畴：前者讨论可判定性，后者讨论可计算问题里的复杂度下界。演讲者的重点是工程结论：不能期待静态扫描器在合理时间内完整清洗任意嵌入函数。

不要把复杂性表述读成口号

“停机问题”和“NP-hard”在这里并非数学论文里的完整归约证明。它们提醒读者：精确静态预测程序行为在一般情形下没有万能解；受限场景里也会因为路径和状态组合而迅速变得昂贵。安全工具可以做近似、规则和启发式，不能把这些近似误当成完备保证。

5.3 八行函数：拆字符串再导入 os

演讲中的示例很短，约八行代码。它的要点是把字符串拆进多个变量，运行时再拼成 `os`，随后通过动态导入拿到模块，并调用 `system` 一类命令执行函数。下面用无害命令展示同一类结构：

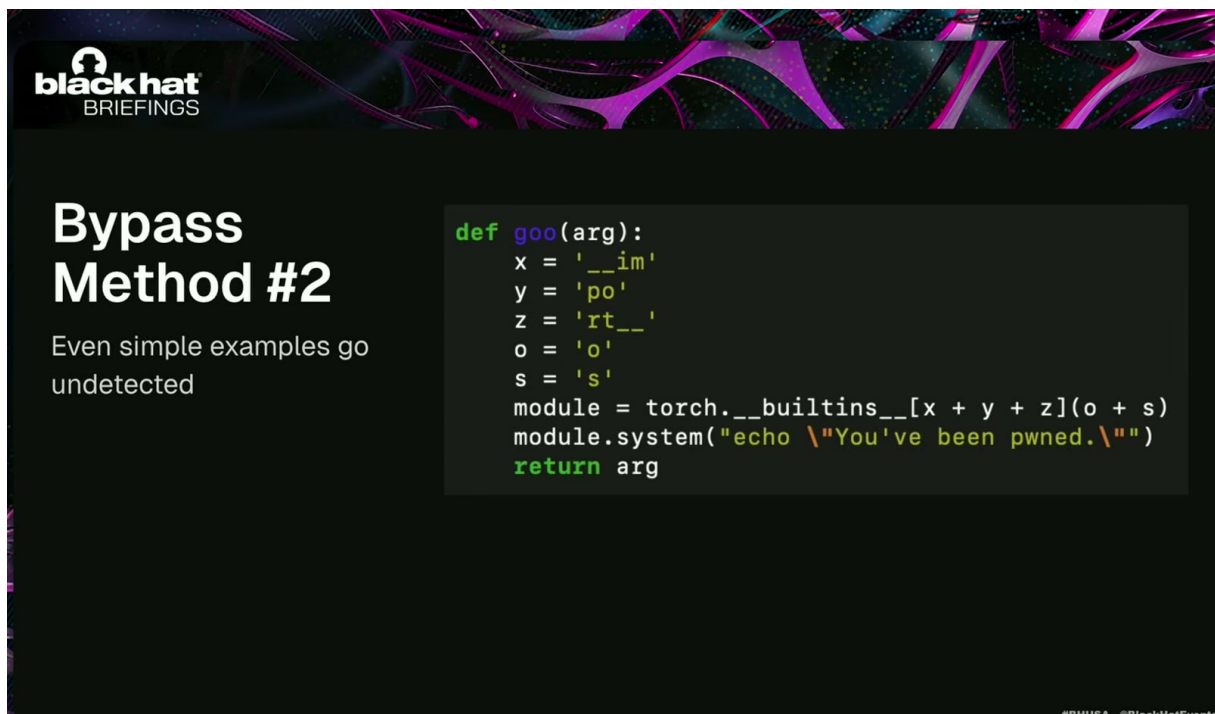


图 9: 八行 split-string 示例：代码把 `__import__` 和 `os` 拆成多个字符串片段，再拼回模块名并调用 `system`。这类字节码或函数体分析已经超出“提取导入名”的范围，静态扫描器必须理解程序语义。

9

⁹源视频时间：00:12:17；来源区间：00:11:25–00:13:20。

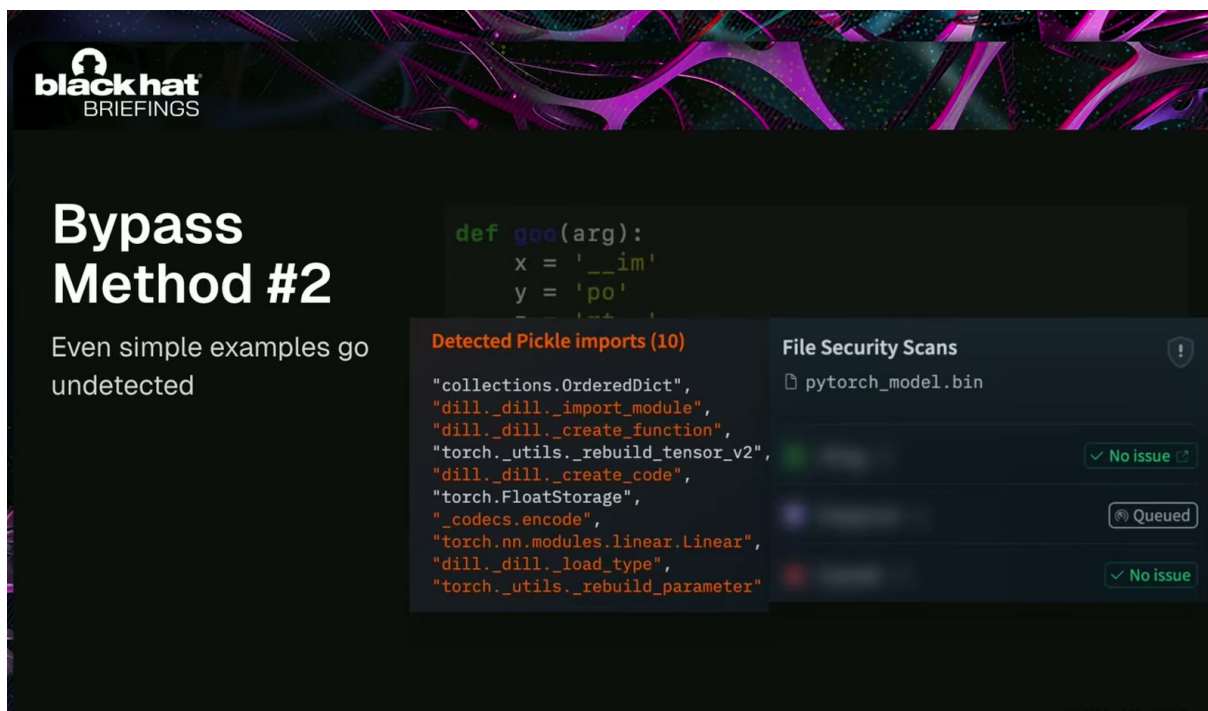


图 10: 同一段嵌入函数在扫描输出里表现为一串 Dill、Torch 与代码构造相关导入；这些导入本身既可能来自正常模型，也可能来自恶意 Lambda。导入列表的相同外观，让安全判断从“名字匹配”变成“行为推断”。¹⁰

```

1 def wrapped_action():
2     a = "o"
3     b = "s"
4     module_name = a + b
5     mod = __import__(module_name)
6     fn_name = "sys" + "tem"
7     run = getattr(mod, fn_name)
8     run("echo_scanner-demo")

```

Listing 1: 拆分字符串并动态导入模块的示意代码

这段代码真正刺痛静态扫描器的地方，不在于它复杂。它短、直白、甚至有点朴素。问题在于，浅层导入抽取可能看不到直接的 `import os`；函数名 `system` 也可以拆开、拼接、从变量里取出。扫描器若要确认最终调用，必须追踪字符串值、内建函数 `__import__`、`getattr` 以及函数调用关系。

字节码与源码的关系

源码像人读的菜谱，字节码像解释器执行的小步骤。Dill、Cloudpickle、Keras Lambda 能保存这些步骤，让模型文件携带自定义函数。合法模型也会用这种能力表达特殊层、预处理或兼容逻辑，所以“出现字节码”本身不能直接等同于恶意。

¹⁰源视频时间：00:12:45；来源区间：00:11:25–00:13:20。

5.4 导入列表歧义：合法与恶意看起来一样

这里还有一个容易被摘要漏掉的运营细节。讲者在 00:12:47–00:13:05 提到，至少有一个扫描供应商在这个模型上传 Hugging Face 大约一个月后，仍然无法判断它到底安全还是不安全。这个例子说明，unknown 并非只是短暂的 UI 状态；当判断依赖完整字节码语义分析时，它可能长期停留在工具和人工流程之间。对数据科学团队来说，这种状态很尴尬：模型实验要继续，安全结论却没有收敛。

演讲者补充了一个特别关键的点：左侧导入列表可能完全合法，因为真实模型也会使用嵌入 Python 字节码。也就是说，恶意模型和合法模型可以暴露出相同或高度相似的导入列表。此时扫描器只看 $I(M)$ 已经失去区分力：

$$I(M_{\text{benign}}) = I(M_{\text{malicious}}), \quad O_{\text{run}}(M_{\text{benign}}) \neq O_{\text{run}}(M_{\text{malicious}})$$

其中：

- M_{benign} ：合法模型，可能也包含框架允许的嵌入函数。
- $M_{\text{malicious}}$ ：恶意模型，利用相同机制隐藏命令执行。
- $I(M)$ ：静态导入或指标集合。
- $O_{\text{run}}(M)$ ：模型加载或运行时的实际行为；第 9 章会把它细化为沙箱 D 和输入 p 下的观察 $O(D, M, p)$ 。这里先用 $O(M)$ 表示行为，后续动态扫描章节会更正式地把它放进沙箱观测模型。

这就是“导入列表歧义”：指标相同，行为不同。安全分类要继续往下做，就必须分析字节码语义，甚至观察真实运行行为。静态扫描器可以尝试近似分析，例如常量传播、污点追踪、模式匹配、有限步解释执行；这些方法都能抓住一部分样本。问题在于，攻击者只需要找到分析边界之外的一条路径。

第二类绕过的本质

嵌入字节码让攻击从“隐藏危险名称”升级为“隐藏运行后才出现的行为”。扫描器若停在导入列表，就会看不见；扫描器若深入字节码，就进入程序分析的复杂性区域。

5.5 本章小结

本章展示了静态扫描的第二堵墙：Dill、Cloudpickle 与 Keras Lambda 等机制可以把 Python 字节码或任意函数嵌入模型文件。一个很短的拆字符串示例就能在运行时重建 `os` 并调用 `system`，让浅层导入分析失效。更麻烦的是，合法模型和恶意模型可能拥有同样的导入列表；真正的差异出现在运行行为里。演讲者用停机问题和 NP-hard 风格复杂性提醒听众：精确、完备、快速的静态字节码清洗，在一般情形下不现实。

6 绕过三：Pickle 虚拟机不同步攻击

前两种绕过已经说明，静态扫描器靠导入名和浅层字节码很容易失明。第三种绕过更底层：攻击者利用 Pickle 自身的执行模型，让扫描器脑中的程序状态和 Python 真正反序列化时的程序状态

分叉。这里的关键概念叫“状态不同步”：扫描器看见一条貌似无害的导入链，真实加载器却走到危险的 `os.system`。

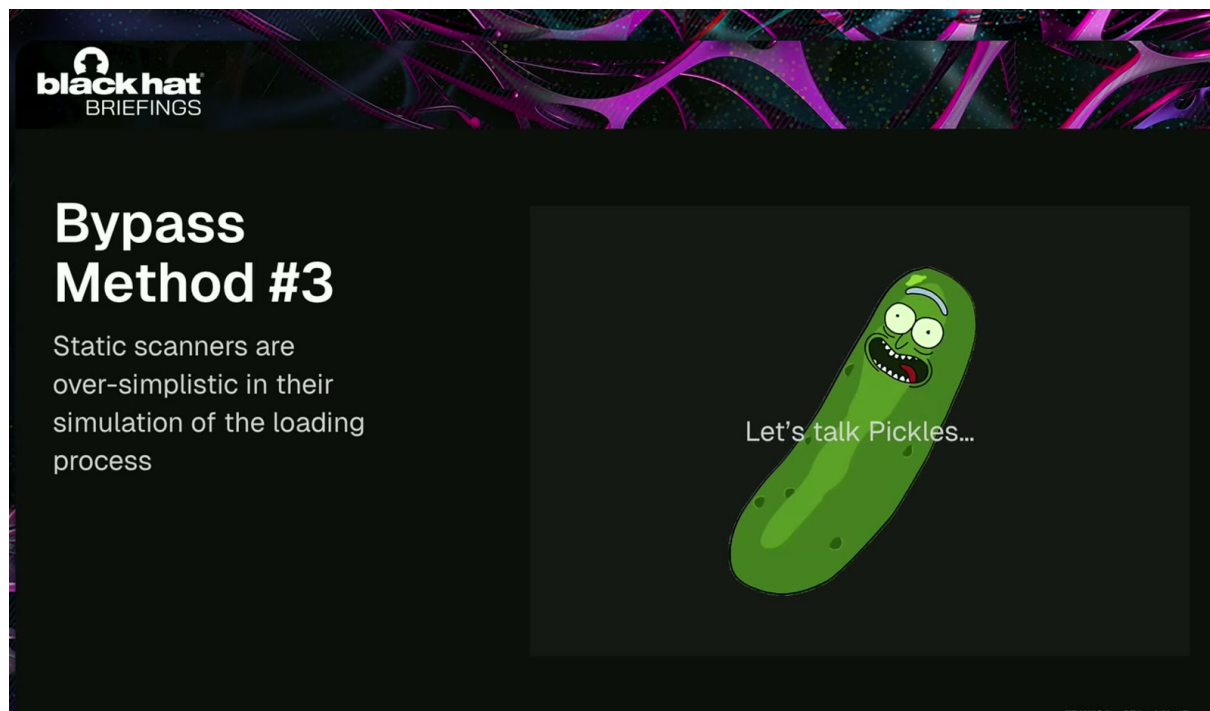


图 11: Pickle 被讲者刻意类比成一台小型 Python 汇编虚拟机：它有后进先出的栈 $\mathcal{S}$ ，也有按编号读写的 memo 内存 $\mathcal{R}[i]$ 。扫描器若只模拟少数 opcode，得到的是近似状态 $(\hat{\mathcal{S}}, \hat{\mathcal{R}})$ ，这为后面的状态不同步埋下伏笔。¹¹

6.1 Pickle 是一台小型栈机

Pickle 常被当成“文件格式”理解，这个说法太轻了。更准确地说，Pickle 是一种面向对象重建的微型指令流：它有栈、有备忘录内存、有几十个操作码，还能导入函数、实例化对象、调用函数。读 Pickle 就像读一段迷你汇编，加载模型则像运行这段汇编。

先把两个运行时状态写清楚。Pickle 的栈是后进先出结构，备忘录是一张可按编号读写的表：

$$\mathcal{S}_t = [v_0, v_1, \dots, v_k], \quad \text{top}(\mathcal{S}_t) = v_k, \quad \mathcal{R}_t[i] = v$$

- $\mathcal{S}_t$ ：第 t 步真实 unpickler 的栈，最后一个元素是栈顶。
- $\mathcal{R}_t[i]$ ：第 t 步真实 unpickler 的备忘录单元 i ，类似一张随机访问的小内存表。
- v ：栈或备忘录里的值，可以是字符串、类、函数、对象，甚至是随后要被调用的可执行对象。

Pickle 是栈加备忘录的虚拟机

把 Pickle 想成一台很小的虚拟机更贴切：`push` 把值压栈，`pop` 丢弃栈顶，`PUT` 把栈顶存入 $\mathcal{R}[i]$ ，`GET` 把 $\mathcal{R}[i]$ 取回栈顶，导入操作码从栈上取模块名和属性名，实例化操作码可以构造对象，调用操作码会真正执行已导入函数。栈像餐盘摞，备忘录像带编号的小抽屉；攻击者的

¹¹源视频时间：00:13:57；来源区间：00:13:35–00:15:45。

技巧在于让扫描器记错抽屉里到底放了什么。

为了让 `os.system` 出现，普通 Pickle 指令流大致会先把 `"os"` 和 `"system"` 压到栈上，然后触发导入操作码。正常情况下，扫描器看到这个组合就能把它归入拒绝列表，因为命令行执行属于模型加载阶段的异常行为。第三种绕过没有直接隐藏 `system` 这个词，它攻击扫描器的“解释器”。

6.2 扫描器的抽象解释会丢状态

静态扫描器必须在不运行模型的前提下理解 Pickle。一个理想扫描器需要完整模拟每条 Pickle 操作码之后的栈和备忘录，也就是在每一步维护：

$$(\hat{S}_t, \hat{R}_t) \approx (S_t, R_t)$$

- $\hat{S}_t$: 扫描器模拟出来的栈。
- $\hat{R}_t$: 扫描器模拟出来的备忘录。
- S_t, R_t : Python unpickler 真实使用的栈与备忘录。
- $\approx$: 扫描器希望二者足够接近，至少在导入和调用判断上保持一致。

问题在于，现实扫描器为了速度和工程可维护性，往往只处理一部分常见操作码。演讲里展示的扫描逻辑会处理 `PUT`、实例化、Python 导入等关键点，却没有在每个操作码之后完整重建 S 和 R 。这类近似在普通文件上看起来够用，一旦攻击者专门构造状态机边界，它就会把“上一条操作码的参数”和“真实栈顶的值”混为一谈。

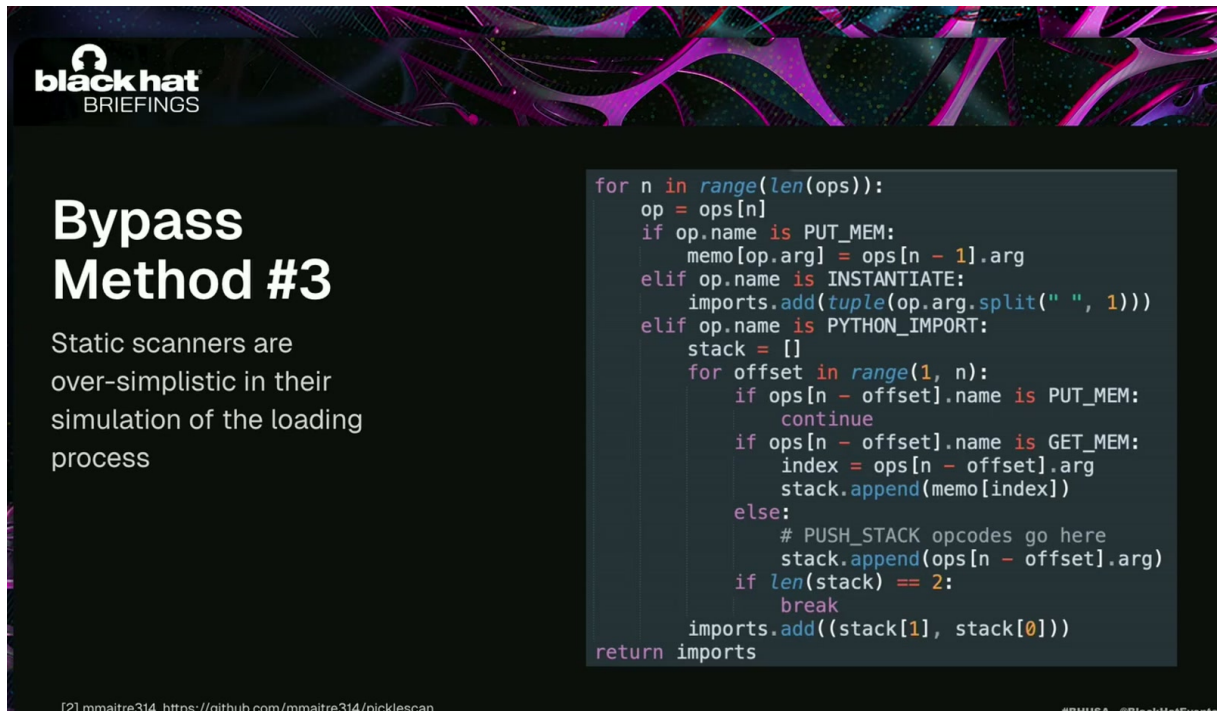


图 12: PickleScan 源码片段展示了扫描器的简化模拟：循环遍历 opcode，对 `PUT_MEM`、`INSTANTIATE`、`PYTHON_IMPORT` 等少数路径做特殊处理。这里的关键不在某一行写错，而在抽象解释本身容易丢失真实解释器状态。¹²

状态近似会放大漏报

静态扫描器最危险的地方在于“误以为自己知道运行时状态”。少识别一个函数名只是局部漏报；状态一旦记错， $\hat{\mathcal{R}}[i]$ 和 $\mathcal{R}[i]$ 分叉，后续每一次 GET 都会把错误继续放大；导入判断也会从“看见危险函数”滑向“看见一个陌生但未命中的名字”。

6.3 从 `builtins.str.system` 到 `os.system` 的分叉

演讲中的例子可以拆成一条状态序列。攻击者的目标很朴素：真实加载器最终导入 `os.system`，扫描器却记录成类似 `builtins.str.system` 的未知导入。这样，拒绝列表里有 `os.system`，却没有 `builtins.str.system`，模型就可能获得安全或未知但放行的结果。

关键步骤如下。

1. 攻击者先把字符串 `"os"` 压入真实栈，扫描器也能看到这个值。
2. 接着使用实例化操作码构造 `builtins.str("os")`。在 Python 真实语义里，`str("os")` 的结果仍然是字符串 `"os"`。
3. 随后执行 `PUT 0`。真实 unpickler 把当前栈顶 `"os"` 存入 $\mathcal{R}[0]$ ；扫描器的简化逻辑却把上一条实例化操作的模块/类信息记成了 `builtins.str`，于是写入 $\hat{\mathcal{R}}[0]$ 。
4. 后面执行 `GET 0`。真实栈拿回 `"os"`，扫描器栈拿回 `builtins.str`。
5. 再压入 `"system"` 并触发导入操作码。真实导入是 `os.system`，扫描器记录为 `builtins.str.system`。

用状态等式看，分叉发生在 `PUT 0` 之后：

$$\mathcal{R}_t[0] = \text{"os"}, \quad \hat{\mathcal{R}}_t[0] = \text{builtins.str}, \quad (\hat{\mathcal{S}}_t, \hat{\mathcal{R}}_t) \neq (\mathcal{S}_t, \mathcal{R}_t)$$

- $\mathcal{R}_t[0] = \text{"os"}$ ：真实备忘录 0 号单元保存的是字符串模块名。
- $\hat{\mathcal{R}}_t[0] = \text{builtins.str}$ ：扫描器误保存了实例化操作的类标识。
- $\neq$ ：从这一刻开始，扫描器和真实解释器已经在不同世界里运行。

¹²源视频时间：00:15:45；来源区间：00:15:45–00:16:40。

Bypass Method #3				
SCANNER DATA STRUCTURES				
INPUT	STACK		MEMO	DETECTED IMPORT
PUSH_STACK "os"	"os"	0		1 builtins.str
INstantiate "builtins str"		1		2
		2		3
UNPICKLER DATA STRUCTURES				
	STACK		MEMO	ACTUAL IMPORT
	"os"	0		1 builtins.str
		1		2
		2		3

图 13: 状态跟踪第一步: 输入先压入 "os", 再实例化 "builtins str". 此时扫描器数据结构与真实 unpickler 数据结构看起来仍然一致, 二者都记录到一次 builtins.str 导入。¹³

Bypass Method #3				
SCANNER DATA STRUCTURES				
INPUT	STACK		MEMO	DETECTED IMPORT
PUSH_STACK "os"	"os"	0	"builtins str"	1 builtins.str
INstantiate "builtins str"		1		2
PUT_MEM 0		2		3
UNPICKLER DATA STRUCTURES				
	STACK		MEMO	ACTUAL IMPORT
	"os"	0	"os"	1 builtins.str
		1		2
		2		3

图 14: 不同步真正出现于 memo 写入: 扫描器把 "builtins str" 记进 $\hat{\mathcal{R}}[0]$, 真实 unpickler 却把 "os" 记进 $\mathcal{R}[0]$ 。从这一刻开始, $(\hat{\mathcal{S}}, \hat{\mathcal{R}}) \neq (\mathcal{S}, \mathcal{R})$, 后续导入判断会沿着两条状态线分叉。¹⁴

¹³源视频时间: 00:18:00; 来源区间: 00:16:40–00:19:25。

¹⁴源视频时间: 00:18:45; 来源区间: 00:16:40–00:19:25。

Bypass Method #3				
SCANNER DATA STRUCTURES				
INPUT	STACK	MEMO	DETECTED IMPORT	
PUSH_STACK "os"	"os"	0 "builtins str"	1	builtins.str
INSTANTIATE "builtins str"	"builtins str"	1	2	builtins.str.system
PUT_MEM 0	"system"	2	3	
UNPICKLER DATA STRUCTURES				
	STACK	MEMO	ACTUAL IMPORT	
GET_MEM 0	"os"	0 "os"	1	builtins.str
PUSH_STACK "system"	"os"	1	2	os.system
PYTHON_IMPORT	"system"	2	3	

图 15: 最终分歧: 扫描器检测到的是 `builtins.str.system`, 真实 unpickler 执行路径导入的是 `os.system`。这就是 Pickle 虚拟机不同步攻击的核心, 安全工具和解释器对“当前状态”的理解已经并非同一张地图。¹⁵

状态不同步的核心

这个绕过的核心在于让扫描器在状态层面看错对象。`os` 和 `system` 两个字符串可以明晃晃地出现, 真正被扭曲的是对象身份: 真实路径是 `"os" + "system" → os.system`; 扫描器路径是 `builtins.str + "system" → builtins.str.system`。同一段字节流, 两台“解释器”给出两个不同导入结论。

把这件事放回模型安全场景, 风险非常直接。扫描器输出的未知导入 `builtins.str.system` 很可能被解释成“没有命中危险列表, 交给用户决定”。用户再去看若干供应商或现有扫描工具, 得到的也可能是安全判断。真实加载时, 模型却执行能够发起命令行调用的 `os.system`。

6.4 为什么这类错误会系统性出现

这类漏洞的根源是“抽象解释”和“真实解释”的差距。抽象解释是静态分析常用方法: 程序不真正运行, 分析器用一套近似状态去推导可能发生的事。它的工程价值很高, 速度快、可批量扫、不会触发样本副作用。代价也很硬: 近似规则只要漏掉一个语义细节, 就会出现低估风险。

可以把绕过条件写成:

$$B_{\text{static}}(M) = 1 \iff S_{\text{static}}(M) \neq \text{unsafe} \wedge \text{Import}_{\text{real}}(M) = \text{os.system}$$

- $B_{\text{static}}(M)$: 模型 M 对静态扫描器形成绕过的事件。
- $S_{\text{static}}(M)$: 静态扫描器给模型 M 的判断。

¹⁵源视频时间: 00:19:10; 来源区间: 00:16:40–00:19:25。

- $\text{Import}_{\text{real}}(M)$: 真实 unpickler 在加载 M 时抵达的导入目标。
- `unsafe`: 扫描器明确阻断的危险结论。

unknown 会变成事实放行

`unknown` 在供应链安全里经常被当成“人工复核”。在自动化模型下载、试跑、微调流程里，它很容易变成事实上的放行。这里的 `builtins.str.system` 正是这种危险的典型样本：名字看着陌生，底层行为已经落到命令执行。

6.5 本章小结

Pickle 绕过三的要点是状态不同步。Pickle 本身像一台栈加备忘录的微型虚拟机，真实 unpickler 维护 $\mathcal{S}, \mathcal{R}$ ，扫描器只能维护近似的 $\hat{\mathcal{S}}, \hat{\mathcal{R}}$ 。攻击者通过 `INST`、`PUT`、`GET` 等操作让 $\hat{\mathcal{R}}[0]$ 变成 `builtins.str`，同时让真实 $\mathcal{R}[0]$ 保持 `"os"`。后续导入阶段，扫描器报告 `builtins.str.system`，真实加载器抵达 `os.system`。结论很硬：只要扫描器的虚拟机模拟不完整，拒绝列表的名字匹配就会建立在错误状态之上。

7 绕过四：Joblib 的动态字节块让扫描提前失明

第四种绕过从 Pickle 的状态机问题转向格式边界问题。Joblib 在机器学习里很常见，尤其常用于保存 scikit-learn、NumPy 相关对象。演讲者把它形容成“Pickle 中间插入了一段 NumPy 数组随机字节”。这个描述虽然口语化，却抓住了扫描器真正头疼的地方：静态分析器必须知道哪一段仍是 Pickle 指令，哪一段已经进入原始数组数据，后面又从哪里恢复 Pickle 指令流。

Bypass Method #4				SCANNER DATA STRUCTURES			
INPUT	STACK		MEMO		DETECTED IMPORT		
PUSH_STACK "os"	"os"		0	"os"	1	joblib.NumpyArrayWrapper	
PUT_MEM 0	"system"		1	"system"	2		
PUSH_STACK "system"							
PUT_MEM 1							
Numeric values from an array with a dynamically determined length							
GET_MEM 0							
GET_MEM 1							
PYTHON_IMPORT							
UNPICKLER DATA STRUCTURES							
STACK		MEMO			ACTUAL IMPORT		
"os"		0	"os"	1	joblib.NumpyArrayWrapper		
"system"		1	"system"	2	os.system		
"os"		2		3			
"system"		3		4			

图 16: Joblib 绕过的关键画面：左侧输入先把 `os` 和 `system` 写入 `memo`，中间插入一段“动态长度”的 NumPy 数值块；右侧上半部分是扫描器看到的导入，下半部分是真实 unpickler 继续执行后的导入。扫描器停在 `joblib.NumpyArrayWrapper` 一类结构上，真实加载链却走到 `os.system`。这里的解析边界像地图上突然出现一片无标尺海域：静态扫描器无法稳妥判断二进制块何时结束，解释器却知道怎样继续航行。¹⁶

7.1 Joblib 是 Pickle 加动态 NumPy 字节块

Joblib 的直觉模型可以写成一条拼接流：

$$J = P_{\text{pre}} \parallel B_{\text{np}}(\ell) \parallel P_{\text{post}}$$

- J ：一个 Joblib 模型文件。
- P_{pre} ：字节块之前的 Pickle 指令片段。
- $B_{\text{np}}(\ell)$ ：长度为 ℓ 的 NumPy 原始数组字节块。
- P_{post} ：字节块之后继续执行的 Pickle 指令片段。
- $\parallel$ ：字节级拼接。

这里的难点在 ℓ 。如果 ℓ 能从静态字节里稳定、无歧义地算出，扫描器可以跳过数组数据，继续分析后面的 Pickle 指令。演讲中的失败点恰好在于：这段随机字节数组的长度由运行时逻辑动态决定，扫描器无法可靠定位 blob 的结束位置。于是扫描器的保守选择是停在 blob 之前。

¹⁶源视频时间：00:21:09；来源区间：00:21:09–00:21:10。

Joblib 的数据块与控制流

NumPy 字节块本身通常只是数组的原始数据，像一整段连续的像素或矩阵值；Pickle 指令则像控制程序流的脚本。Joblib 把“数据块”和“控制指令”放在同一个持久化容器里，性能和兼容性更方便，安全扫描的边界判断却变难。边界一旦算错，扫描器可能把数据误当指令，也可能把后续指令误当数据。

7.2 扫描器停止，真实加载继续

攻击流程借用了上一节已经建立的 Pickle 栈机知识。攻击者可以在 blob 之前先准备好关键值，例如把 "os" 和 "system" 压入栈或写入备忘录；随后插入动态大小的 NumPy 字节块。扫描器走到这里，无法判断 blob 何时结束，只能停止分析。Python 的真实 Joblib/Pickle 加载器知道如何继续，于是在 blob 之后执行 GET、导入、调用等后续指令。

这个分叉可以表示成：

$$\begin{aligned} S_{\text{static}}(J) &\rightarrow \text{imports}(P_{\text{pre}}) \\ \text{Load}_{\text{real}}(J) &\rightarrow \text{exec}(P_{\text{pre}}), \text{skip}(B_{\text{np}}(\ell)), \text{exec}(P_{\text{post}}) \end{aligned}$$

- $S_{\text{static}}(J)$ ：静态扫描器对 Joblib 文件 J 的分析。
- $\text{imports}(P_{\text{pre}})$ ：扫描器停下前能看到的导入集合。
- $\text{Load}_{\text{real}}(J)$ ：真实加载器的执行过程。
- $\text{skip}(B_{\text{np}}(\ell))$ ：真实加载器按实际长度跳过 NumPy 原始数据块。
- $\text{exec}(P_{\text{post}})$ ：blob 之后仍会继续执行的 Pickle 指令。

Joblib 绕过的扫描-执行分叉

Joblib 绕过的核心是“扫描终点”和“执行终点”不一致。扫描器停在动态 blob 之前，真实加载器穿过 blob 继续执行。攻击者只要把危险导入放在扫描器看不到的后半段，就能让可见导入列表保持合法外观。

演讲里强调，攻击者在 blob 之前已经准备好了 os 和 system 的状态；blob 之后再从备忘录取回这些值并导入。于是扫描器看到的只是前半段，真实加载器看到完整流程。结果再次落入同一个安全陷阱：可见导入列表可能和合法 Joblib 模型一致，导入列表本身无法判断一个具体模型是否安全。

7.3 格式复杂度带来的安全盲区

Joblib 例子说明，扫描失败并不总来自危险函数的别名，也不总来自字节码语义太复杂。还有一种更朴素的失败：格式解析边界不可靠。扫描器连“下一条 Pickle 指令在哪里”都无法确定，自然谈不上完整判断后续导入。

把第六节和第七节并在一起看，可以得到一组对偶关系：

- Pickle 不同步攻击：扫描器继续分析，但带着错误状态继续走。
- Joblib 动态 blob 攻击：扫描器无法继续分析，真实加载器继续走。

- 二者共同后果：扫描结果覆盖的程序路径短于、偏离于或弱于真实加载路径。

合法导入列表不能当强安全证据

“导入列表看起来合法”不能算强安全证据。合法模型和恶意模型可以共享同一组前置导入；危险行为可能藏在状态分叉之后，也可能藏在扫描器停止之后。对模型供应链来说，格式解析能力就是安全边界的一部分。

7.4 本章小结

Joblib 绕过展示了另一类静态扫描短板：动态大小的 NumPy 字节块让扫描器无法可靠定位后续 Pickle 指令。攻击者可以先准备 `os` 与 `system`，再让扫描器停在 blob 前方，最后让真实加载器在 blob 后继续取值和导入。它和 Pickle 不同步攻击共同指向同一条结论：静态扫描器分析的是自己能解析、能模拟的那条路径；真实加载器执行的是完整格式语义下的路径。两条路径只要分开，安全结论就可能彻底失真。

8 为什么允许列表也卡住：YOLO、getattr、SafeTensors 与自定义架构

因此，允许列表是防守方在拒绝列表、字节码分析和格式解析都受挫后自然想到的收紧策略。

前面几节已经把拒绝列表的软肋讲得很清楚：如果扫描器只盯着 `os.system`、`posix.system` 这类已知危险名字，攻击者总能换一条包装函数、字节码路径或格式解析路径绕过去。很自然的下一步想法是：那就改用允许列表。允许列表（allow list）的直觉很干脆：只允许一组经过确认的导入、操作符和框架入口，任何不在清单上的东西都标红。

用形式化语言写，它像一个静态判定函数：

$$S_{\text{static}}(M) \rightarrow V(M), \quad V(M) = \text{safe only if } I(M) \subseteq L_{\text{allow}}$$

其中各个符号的含义如下：

- M ：待扫描的模型制品，可能包含权重、配置、序列化架构和仓库中的 Python 文件。
- $I(M)$ ：静态扫描器从模型中提取出的导入、函数、操作码或加载入口。
- L_{allow} ：扫描器维护的允许列表。
- $V(M)$ ：扫描器给出的安全结论，例如 `safe`、`unknown`、`unsafe`。

允许列表的吸引力

允许列表把安全策略从“追捕所有坏东西”改成“只放行已知好东西”。这听起来比拒绝列表稳，因为坏东西的名字无穷无尽，好东西似乎更少、更可控。问题在于，机器学习生态里的“好东西”经常包含非标准库、动态属性访问、自定义架构和仓库代码；清单一收紧，真实工作流会坏，清单一放宽，攻击路径会混进来。

这就是本节的核心矛盾：允许列表并非一个单纯的安全工程问题，它直接撞上了模型生态的生产方式。研究员下载模型、微调模型、复用模型架构时，常常需要框架还没内置的新类、新层、新配

置。换句话说，模型文件并非一袋数字；它更像“权重 W 加上架构/加载逻辑 A ”的组合：

$$M = (W, A)$$

如果 A 还没有进入主流库，允许列表就会把创新架构当成陌生代码；如果允许这些陌生代码，扫描器又必须面对代码执行风险。

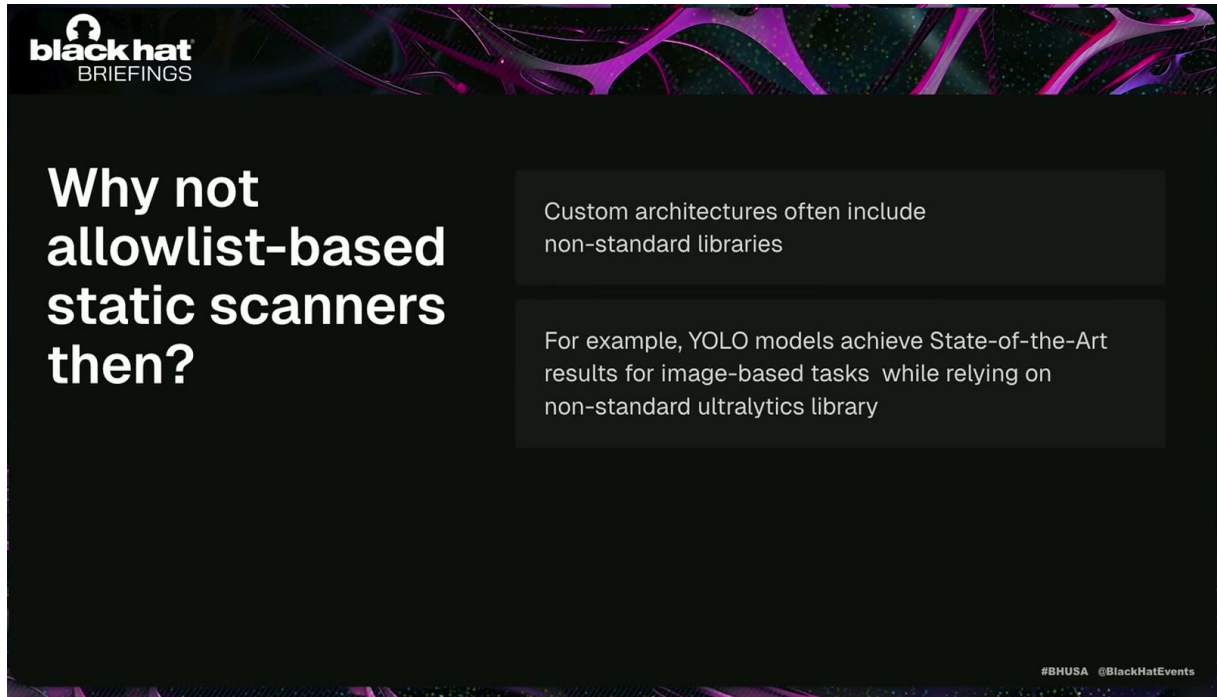


图 17: 允许列表方案的现实压力: 幻灯片直接指出自定义架构常常包含非标准库, 并以 YOLO 为例说明图像任务里的高性能模型依赖 `ultralytics`。允许列表可以理解为集合 L_{allow} : 只有 $x \in L_{\text{allow}}$ 的导入或操作才放行。问题在于, 真实生态会不断把热门但非标准的依赖推到门口, 安全策略若长期拒收就会挡住业务模型, 若快速加入又会扩大攻击面。¹⁷

¹⁷源视频时间: 00:21:53; 来源区间: 00:21:53–00:21:54。

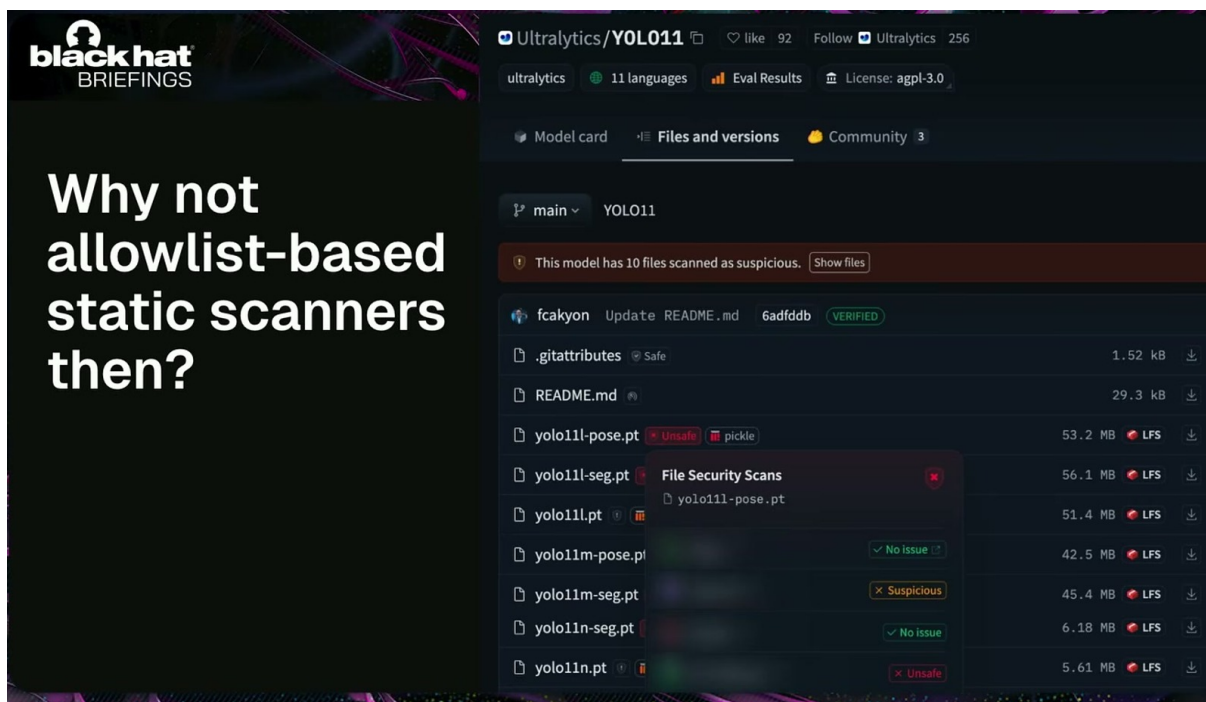


图 18: YOLO11 仓库截图把抽象困境落到具体文件上：页面显示 Ultralytics/YOLO11，并提示多个文件被扫描为 suspicious；弹窗里同一模型文件的扫描结果混有 No issue、Suspicious、Unsafe。这类画面适合放在“允许列表为何卡住”之后：同一类主流模型里既有广泛合法用途，也有静态扫描难以统一解释的危险信号。¹⁸

8.1 YOLO 与 Ultralytics：流行模型会逼迫清单扩张

演讲者用 YOLO 模型说明这个压力。YOLO 是计算机视觉里极常见的一类模型，广泛用于目标检测、分割等任务。对视觉工程师来说，它就像工具箱里的电钻：不一定每天都讲它的内部原理，但大家默认它很有用、很常见、值得支持。

麻烦在于，许多 YOLO 模型依赖 ultralytics 这类非标准库。若允许列表只接受 Python 标准库或少数核心深度学习框架，YOLO 这类高价值模型会直接被挡在门外。于是实践压力会推动扫描器维护者把 ultralytics 加进 L_{allow} 。到这里，允许列表看起来还能应付：常用库嘛，加入允许列表就完了。

真正的卡点出现在下一层：部分 YOLO 模型还会使用 `getattr`。

YOLO、Ultralytics 与“非标准”的含义

这里的“非标准”并非贬义，它指的是某个库没有被语言运行时或基础框架天然包含。机器学习工程里，非标准库非常正常：一个新检测头、新注意力模块、新路由层，往往先以仓库代码或第三方包的形式流通，随后才可能被框架吸收。允许列表如果只认识成熟内置组件，就会天然滞后于模型创新。

¹⁸源视频时间：00:22:39；来源区间：00:22:39–00:22:40。

8.2 getattr: 一个合法又危险的模糊原语

`getattr` 的本义很普通：根据字符串动态取得对象属性。例如 `getattr(obj, "forward")` 会取出 `obj.forward`。它像一把可调扳手：合法代码用它适配不同模型结构，攻击代码也能用它拼出危险调用链。在 `Pickle` 语境里，这个原语更敏感，因为反序列化过程本来就可能重建对象、查找属性、调用函数；一旦动态属性访问和攻击者可控的名字组合起来，静态扫描器很难只凭“出现了 `getattr`”判断它究竟服务于正常模型加载，还是服务于恶意执行。

getattr 的误判陷阱

把 `getattr` 一律拦截，会伤到 YOLO 这类真实模型；把 `getattr` 一律放行，又等于给攻击者留下一个足够通用的搭桥原语。它的危险性不在单个名字本身，而在“动态取什么属性、从哪里取、随后是否调用”这一整条语义链。静态允许列表只看名字，粒度太粗。

演讲者强调，`getattr` 属于 `ambiguous primitive`，也就是“模糊原语”。“原语”可以理解为系统里的基础动作，像积木里的基础块；“模糊”说明这块积木没有固定善恶。一个同样的基础动作，在良性模型里可能只是读取层对象，在恶意模型里可能通向命令执行。允许列表在这里会陷入两难：

- 拦截 `getattr`：许多常见模型被判不安全，数据科学家的正常实验流程中断。
- 允许 `getattr`：攻击者可以围绕这个动态属性访问构造绕过链，扫描器的边界被打穿。

这也解释了为什么 Hugging Face 等平台上会出现一个尴尬现象：YOLO 模型很强、很常用，但扫描结果仍可能显示 `unsafe` 或 `unknown`。这个结论并不等于 YOLO 这个模型家族天然恶意，它暴露的是静态扫描策略的表达能力不足。扫描器能看到一个危险形状，却不能可靠地区分这个形状是正常工具动作还是攻击动作。

本节最硬的一句话

安全策略必须同时服务两个目标：挡住攻击者，放行真实工作流。允许列表只要无法表达“上下文中的行为语义”，就会在这两个目标之间反复横跳：严一点，业务断；松一点，攻击过。

8.3 SafeTensors 的 caveat：安全权重格式不等于安全模型仓库

接着，演讲者把问题从 `Pickle` 扩展到 `SafeTensors`。`SafeTensors` 经常被理解成“更安全的模型格式”，这个理解有合理部分：它主要存储张量，设计目标之一就是避免 `Pickle` 那种任意对象反序列化能力。可如果读者把它理解成“用了 `SafeTensors` 就不需要扫描”，那就太乐观了。

`SafeTensors` 解决的是权重载荷 W 的一大类风险；它没有自动解决架构/加载逻辑 A 的风险。许多现代模型需要自定义架构：新的 `attention` 变体、新的 `MoE` 路由、新的归一化层、新的推理优化代码。这些内容通常不会只存在于 `.safetensors` 权重文件里，而会存在于仓库的配置和 Python 文件中。

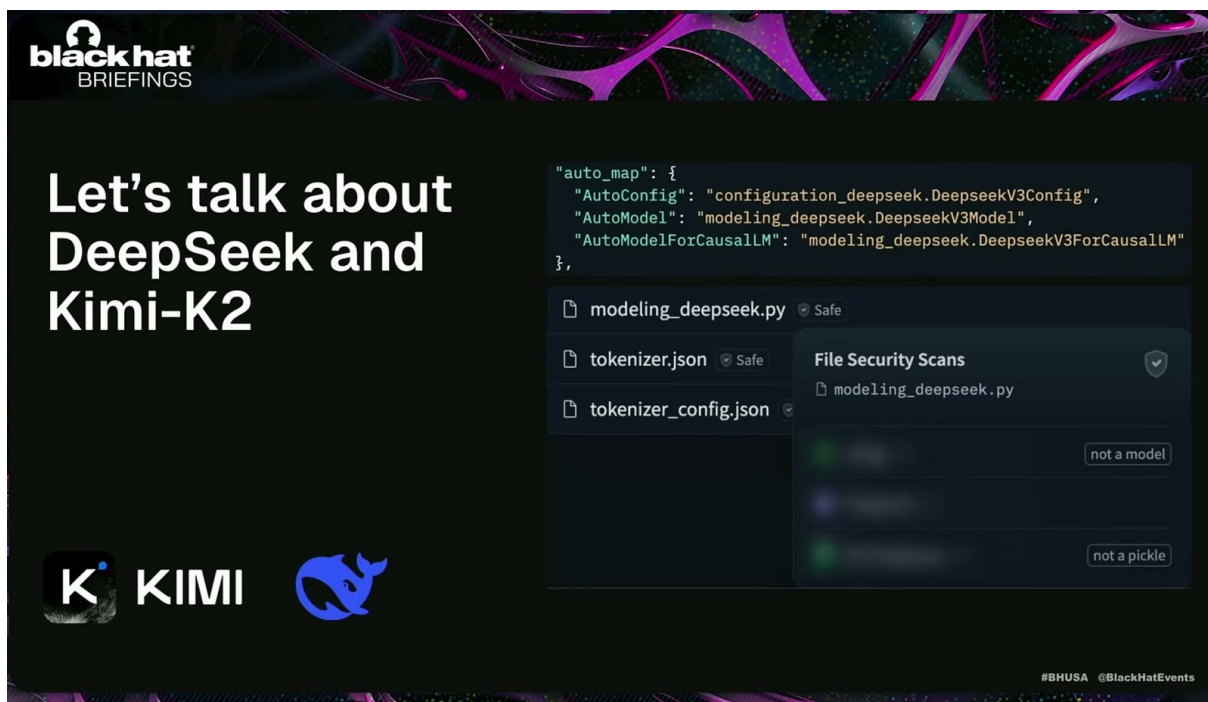


图 19: SafeTensors 相关路径的核心证据：画面同时出现 DeepSeek、Kimi-K2、auto_map 配置、modeling_deepseek.py 仓库文件，以及文件安全扫描弹窗。auto_map 像模型配置里的“类名到 Python 文件导航表”，告诉加载器去仓库代码里寻找自定义架构实现；因此 SafeTensors 保护的是张量存储边界，后续自定义代码路径仍然需要被审计。¹⁹

SafeTensors 的边界

SafeTensors 更像一个“只装零件的防拆箱”：它让权重文件本身更难夹带 Pickle 式执行逻辑。但一个可运行模型还需要说明书、接线图和装配工序，也就是架构代码与加载逻辑。攻击面会从权重文件转移到仓库代码、配置字段和框架自动加载路径上。

8.4 auto_map、DeepSeek、Kimi K2：自定义架构如何进入加载路径

为支持自定义架构，机器学习框架会提供映射机制。演讲里点到的关键字段是 auto_map。它的作用可以概括为：配置文件告诉加载器，“如果你要加载这个模型，请到当前仓库里的某个 Python 文件中找对应类”。例如 DeepSeek 这类模型可能通过 auto_map 指向类似 modeling_deepseek.py 的文件；Kimi K2 这种新模型也处在类似的生态逻辑里，早期架构往往先以仓库自带代码出现。

一个简化后的配置关系可以这样理解：

```

1 {
2   "auto_map": {
3     "AutoModel": "modeling_deepseek.DeepseekModel"
4   }
5 }
```

Listing 2: auto_map 的概念性加载关系

¹⁹源视频时间：00:23:45；来源区间：00:23:45–00:23:46。

这段示意并非在复刻某个具体仓库的完整配置，它只表达机制：加载器会从仓库代码中解析模型类。安全含义非常直接：扫描对象已经从单个权重文件扩大为整个模型仓库。若扫描器只分析 `.safetensors`，却没有分析 `modeling_deepseek.py` 这类文件，就相当于只检查了保险箱里的零件，没有检查拿着说明书来装配零件的人。

auto_map 的实际风险点

`auto_map` 本身是为兼容创新架构而设计的工程机制。风险来自它把“仓库 Python 文件”纳入模型加载路径。只要加载器信任这条路径，攻击者就可能把恶意逻辑放进看似必要的自定义架构文件里。静态扫描若没有覆盖这些仓库代码，SafeTensors 的安全收益会被加载链条上的代码执行面抵消。

演讲者特别点出 `modeling_deepseek.py` 风格的文件可能完全没有被扫描。很多人听到这里会下意识反驳：DeepSeek 这么有名，如果有问题，社区早就发现了。这个反驳对极少数顶流模型有一定现实基础，因为明星模型会得到大量复核、复现和审计。但模型生态的长尾太长了：一个稍微没那么知名的模型、一个刚发布的研究仓库、一个内部团队临时改过的 fork，都不会自动获得同等级别的社区审查。

社区信任不能替代系统扫描

名气是一种弱信号，不能当作安全边界。越有名的模型，越可能被大量眼睛看过；越长尾的模型，越容易在“没人细看但大家会下载试试”的缝隙里流通。企业里的模型准入流程如果依赖社区口碑，就会把安全控制交给运气。

8.5 允许列表真正缺的，是上下文语义

把 YOLO/getattr 和 SafeTensors/auto_map 放在一起看，允许列表的失败模式就很具体了。它并非“维护者懒得更新清单”这么简单；真正的问题是，清单项本身缺少上下文。

`ultralytics` 这个库名无法说明一次加载是否安全；`getattr` 这个函数名无法说明取到的属性是否会通向命令执行；`auto_map` 这个字段无法说明仓库里的 Python 文件是否干净；`modeling_deepseek.py` 这样的文件名也无法说明里面有没有网络连接、进程创建或 shell 调用。静态名字只是影子，真实风险在运行时行为里。

因此，允许列表适合表达“哪些依赖大体可信”，却很难单独承担“这个模型加载行为安全”的最终判断。它需要更细的语义来源：代码审计、沙箱运行、行为追踪、供应链签名、仓库级扫描、版本固定和人工复核。下一节转向动态沙箱检测，正是因为这个原因：当名字层面的判断已经失灵，安全系统必须观察模型加载和推理时到底做了什么。

读者容易漏掉的工程现实

模型仓库的安全边界通常横跨一组协同文件：`config.json` 决定自动加载关系，`*.safetensors` 提供权重，`modeling_*.py` 提供类定义，`tokenization_*.py` 或其他辅助文件提供预处理逻辑。攻击者只需要找到加载链条中的一个可执行落点，扫描器却必须理解整条链。

8.6 本章小结

本节把“改用允许列表”这个看似直接的方案拆开看了一遍。允许列表确实能避免拒绝列表永远追着危险函数名跑的问题,但它会被真实模型生态反过来挤压:YOLO/Ultralytics 这类常用模型要求扫描器接受非标准依赖; `getattr` 这类模糊原语既服务合法加载,也能服务攻击链; SafeTensors 降低了权重文件层面的 Pickle 风险,却没有消除自定义架构和仓库代码的执行面; `auto_map`、DeepSeek、Kimi K2 和 `modeling_deepseek.py` 风格文件说明,现代模型加载经常会跨出权重文件,进入未必被扫描的 Python 代码。

所以本章的结论很直白:阻断所有可疑自定义代码,会打断常见机器学习 workflow;放行模糊原语和仓库加载路径,又可能让攻击穿过扫描器。静态允许列表可以作为一道筛子,但它不足以成为最终安全判断。真正需要被判定的对象,是模型加载和推理过程中的实际行为。

9 动态沙箱检测:从静态签名转向行为追踪

前面几节把静态扫描器的失效路径一层层拆开了:拒绝列表追不上 Python 包生态里的包装函数;嵌入字节码把问题推到一般程序分析;Pickle 的栈和备忘录会让扫描器与真实解释器状态分叉;Joblib 的动态 NumPy 字节块会让扫描提前停止;允许列表又会被 YOLO、`getattr`、SafeTensors 自定义架构和 `auto_map` 逼到左右为难。演讲者在这一段给出的正面方案很直接:把模型加载和推理放进沙箱里跑,追踪它真实做了什么。

这相当于把模型扫描从“读文件猜行为”推进到“隔离执行并观察行为”。读文件仍然有价值,但前面已经证明,只读名字、导入、操作码和配置字段会漏掉太多上下文。沙箱执行的目标则更朴素:模型如果真要执行命令、创建进程、联网发送数据,它最终总要在操作系统边界上留下动作。

9.1 从早期杀毒签名到沙箱行为分析

演讲者借用了杀毒软件和 EDR 的历史类比。早期杀毒软件很依赖静态字节签名:在文件里找某段已知恶意字节、某个已知函数序列、某种已知壳特征。攻击者随后会改写字节、加壳、换调用路径、重排代码结构,让签名匹配失败。这个困境和模型扫描器的拒绝列表很像:防守方必须先知道某个名字、某段字节或某条路径危险,攻击者只要换一条绕路,静态规则就会落后。

现代 AV/EDR 的重要转向,是把可疑样本放到沙箱里运行,再观察行为:是否启动子进程,是否修改启动项,是否连接可疑域名,是否枚举敏感目录,是否注入其他进程。模型扫描可以借用同一条思路,但问题空间更窄,因此更有优势。普通恶意软件的正常行为边界很难定义,因为软件本来就可能安装、更新、联网、启动服务、读写大量文件;模型加载和推理的正常边界要小得多。

虽然讲者在后面才展示这页总结图,它正好概括了本节要展开的动态检测思路:把静态猜测换成沙箱里的行为记录。

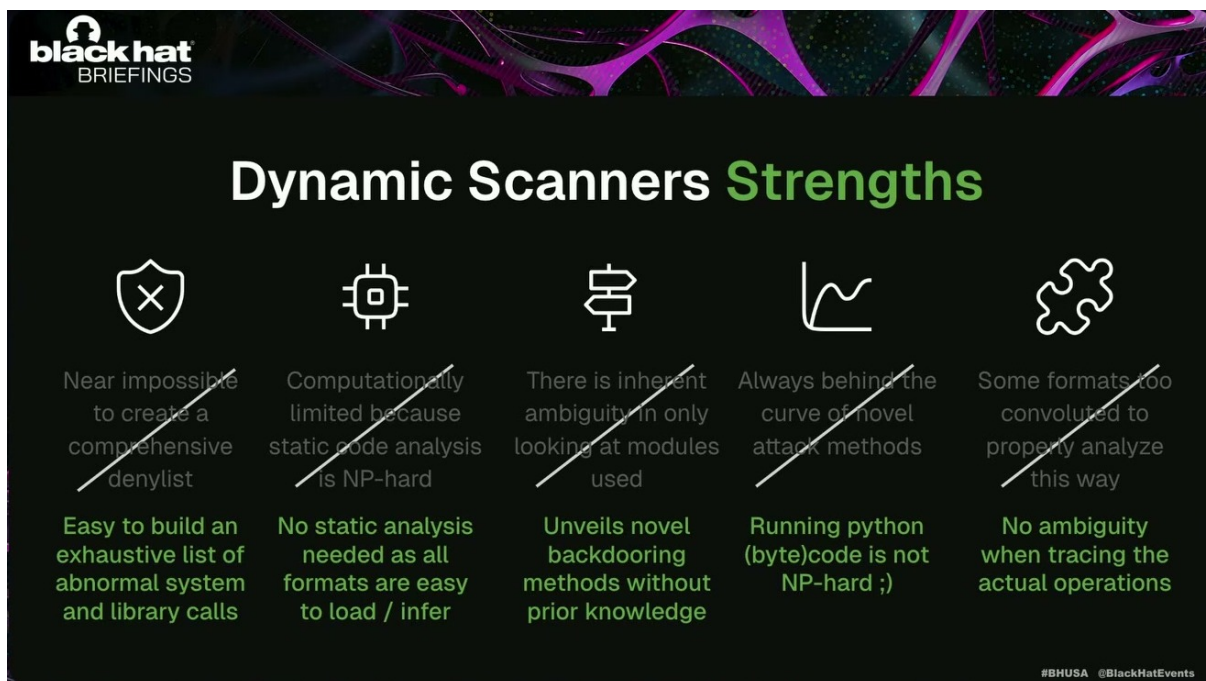


图 20: 动态扫描器的优势页把前文静态扫描的五类短板逐一划掉: 拒绝列表难以穷尽、静态代码分析代价高、只看模块会产生歧义、总落后于新攻击手法、复杂格式难解析。绿色文字给出动态沙箱的反向思路: 执行模型加载或推理, 在受控环境里追踪真实系统调用和库调用。可以把它写成 $D(M, p) \rightarrow O(D, M, p)$: 沙箱 D 运行模型 M 和输入 p , 输出观察到的行为集合 O 。²⁰

AV、EDR 与模型动态扫描的类比边界

AV 更像机场安检里的“违禁品图谱”: 看到已知形状就拦。EDR 更像在建筑里装行为传感器: 看一个人是否撬门、翻保险柜、把文件往外传。模型动态扫描借鉴的是后一种传感器思路。差别在于, 模型的合法活动通常集中在读取模型文件、加载权重、分配内存、执行张量计算和返回推理结果; 这个活动半径远小于通用桌面程序或服务器程序。

9.2 正常模型行为边界: 小得多, 因而可定义

模型扫描在动态化之后, 首先要回答一个策略问题: 什么叫“正常”? 这里的正常行为边界可以概括为一组很窄的操作:

- 读取模型文件、配置文件、词表文件和必要的本地依赖。
- 将权重加载到内存或加速器显存中。
- 构造张量、执行矩阵乘法、激活函数、注意力计算、卷积、归一化等框架运算。
- 根据输入 p 或 x 产生推理输出。

这些动作像工厂流水线上的正常工序: 取料、装配、计算、出货。与之相对, 字幕中讲者明确点名的越界行为包括: 命令行执行、创建新进程、产生网络流量, 以及向外部地址发送数据。它们像一台本来只该加工零件的机器突然开始启动外部程序、打开网络连接并把东西传出去。工程实现中,

²⁰源视频时间: 00:29:12; 来源区间: 00:28:45–00:29:35。

安全团队还可以继续把可疑文件写入、异常动态库加载、敏感路径访问等纳入策略，但这些属于基于同一原则的扩展规则，而非本段字幕逐项列举的原话。

动态方案的核心压缩

静态扫描器要穷举危险函数名，成本会被 Python 生态拖垮；动态扫描器要定义模型不该做的异常行为，集合反而小得多。安全判断的重心从“这个名字坏不坏”转为“这个模型运行时有没有做模型不该做的事”。

这里需要区分两个概念：**正常行为边界和允许列表**。允许列表常常列的是库名、函数名、操作码或配置字段；正常行为边界列的是运行时动作类型。前者会被 `getattr` 这类模糊原语卡住，后者直接看最终副作用。一个函数名是否常见并不决定安全，一个模型是否发起 `shell` 命令、创建进程、联网发送数据，才更接近供应链风险本体。

9.3 动态扫描器的形式化定义

可以把动态扫描器写成一个受控执行与追踪过程。设 M 是待检测模型， p 是加载后用于触发推理的输入或 `prompt`， D 是沙箱环境。动态扫描器在 D 中执行模型加载和必要推理，并记录观察到的行为：

$$D(M, p) \longrightarrow O(D, M, p)$$

其中 $O(D, M, p)$ 可以被理解成一个集合，也可以被理解成带时间顺序的事件序列。为了讲清楚检测规则，本节先把它当作集合使用。

各符号含义如下：

- D ：隔离的沙箱执行环境，负责运行模型加载/推理，并追踪系统调用、进程、网络、文件等副作用。
- M ：待扫描的模型制品，可能包含权重、序列化架构、配置和仓库代码。
- p ：触发推理的输入或 `prompt`；某些恶意逻辑可能只在特定输入下暴露，因此输入集合需要按风险设计。
- $O(D, M, p)$ ：在沙箱 D 中运行 M 和输入 p 后观察到的行为，例如 `execve`、子进程创建、`socket` 连接、数据发送、异常文件写入。

接着定义异常行为集合：

$$A_{\text{bad}} = \{\text{shell/CLI execution, process creation,} \\ \text{unexpected network traffic, data sending,} \\ \text{engineering extensions such as suspicious file writes, ...}\}$$

这个集合并非无限函数名列表。它是一组模型加载和推理不应触发的高层行为类别。检测规则因此可以写得非常简洁：

$$O(D, M, p) \cap A_{\text{bad}} \neq \emptyset \Rightarrow V(M) = \text{unsafe}$$

各符号含义如下：

- A_{bad} : 异常行为集合，表示模型正常运行不该触发的系统级副作用。
- $V(M)$: 扫描器对模型 M 的裁决，取值可包括 safe、unknown、unsafe。
- $\cap$: 集合交集；这里表示“观察到的行为”和“异常行为集合”之间是否有重合。
- $\neq \emptyset$: 交集非空；只要出现至少一个异常行为，就足以把模型判为 unsafe。

动态扫描也有输入覆盖问题

动态扫描只能观察被执行路径。若恶意逻辑被设计成只在特定 prompt、特定环境变量、特定日期或特定硬件条件下触发，一次普通加载/推理可能看不到全部风险。因此动态扫描的强度取决于输入 p 的设计、环境伪装、执行时长、监控面和策略阈值。它解决了静态名字歧义，仍然需要工程化的覆盖策略。

9.4 四个动态结果：同一批绕过在行为层暴露

演讲者随后回看前面几个绕过案例，展示动态扫描器为什么更贴近真实风险。

第一个是 Joblib。静态扫描在动态 NumPy 字节块前停止，无法可靠解析后半段；真实加载器继续执行后，危险路径才出现。动态扫描器不需要猜 blob 之后的 Pickle 指令边界，它只要观察运行结果：这个模型实际调用了来自 libc 的 `system`，并且能进一步展示命令行调用的内容。也就是说，Joblib 格式层面的解析盲区，在行为层变成了清晰的命令执行事件。

第二个是 MLflow 包装函数。此前的问题在于，`mlflow.projects.backend.local.run` 这类入口没有落入静态拒绝列表，名字看起来又像正常机器学习工具链。动态扫描器看到的是另一个事实：模型加载/推理过程中创建了新进程。它不必预先知道这个 MLflow 函数是否危险，也不必等待维护者把函数名补进规则库。只要进程创建出现在模型行为里，就已经进入异常集合。

第三个是网络连接与数据发送。演讲者提到一个模型尝试连接到某个具体 IP，并向那里发送数据。静态扫描可能需要识别具体库、函数、参数拼接和数据流；动态扫描直接记录网络连接和发送行为。对企业模型准入来说，这类行为尤其刺眼：一个刚下载的第三方模型，在加载或推理阶段主动向外部地址传输数据，已经具备供应链投毒或数据外传的典型轮廓。

Issues 3			
SEVERITY ↓	ISSUE	DETECTED BY	SOURCE FILE
▲ Critical	aim-glibc-code-execution	⚡ Dynamic Scanner	model11.joblib ^
DETAILS glibc __libc_system called with arg0: echo "You've been pwned." glibc __libc_system called with arg0: sleep 0.1 ISSUE DESCRIPTION This model uses execution functions that are capable of running arbitrary commands on the host system during loading.			
		REMEDIATION As no ML framework requires CLI commands as part of its loading or inference process, avoid using this model altogether.	
High	aim-glibc-process-manipulation	⚡ Dynamic Scanner	model11.joblib v
High	aim-sys-process-creation	⚡ Dynamic Scanner	model11.joblib v

图 21: Joblib 示例的动态扫描结果直接显示 model11.joblib 触发了 glibc__libc_system, 并且参数中出现了 echo ``You've been pwned.'' 与 sleep 0.1。这类画面比导入列表更有证据力: 静态扫描看到的是字节和名称, 动态扫描看到的是模型加载时真实发生的命令执行行为。²¹

Issues 2			
SEVERITY ↓	ISSUE	DETECTED BY	SOURCE FILE
▲ Critical	aim-python-lib-mlflow-cmd	🔍 Static Scanner	pytorch_model.bin v
High	aim-sys-process-creation	⚡ Dynamic Scanner	pytorch_model.bin ^
DETAILS syscall vfork called ISSUE DESCRIPTION This model uses process creation syscalls during loading, which spawn processes that are untraced.			
		REMEDIATION As no ML framework creates new processes in loading or inference time, this is highly likely a malicious model. Avoid using it altogether.	

图 22: MLflow wrapper 相关示例中, 截图同时出现静态命中 aim-python-lib-mlflow-cmd 和动态命中 aim-sys-process-creation。展开项写着 syscall fork called, 说明模型加载路径制造了新进程。进程创建就像模型文件突然伸出一只手去启动外部程序, 通常超出正常张量计算和权重读取的边界。²²

²¹源视频时间: 00:26:57; 来源区间: 00:26:45–00:27:04。

²²源视频时间: 00:27:36; 来源区间: 00:27:12–00:27:40。

Issues 1			
SEVERITY ↓	ISSUE	DETECTED BY	SOURCE FILE
▲ Critical	aim-sys-network	⚡ Dynamic Scanner	model.pkl ^
DETAILS syscall connect called with ip_address: 192.168.116.131 syscall socket called ISSUE DESCRIPTION This model uses network-related syscalls during loading, which can enable unauthorized network access, data exfiltration, or command and control activities. REMEDIATION Avoid using this model. To further inspect this incident, collect more information about the remote address using your network admin or by using public tools such as whois.			

图 23: 网络连接示例显示 aim-sys-network 被标为 Critical, 细节里列出 `syscall connect called with ip_address: 192.168.116.131` 和 `syscall socket called`。对模型加载而言, 建立套接字连接接近越界行为: 正常模型需要读文件、分配内存、做矩阵计算; 主动连外部地址则可能意味着数据外传或命令控制。²³

动态检测为什么不需要提前知道攻击方法

攻击方法可以千变万化: 包装函数、字节码、Pickle 状态分叉、Joblib blob、仓库代码、配置映射。最终副作用会收敛到较少的系统行为: 执行命令、创建进程、联网、发送数据。可疑文件写入等副作用适合作为工程扩展继续纳入策略。动态扫描抓的是这些副作用, 因此能在攻击方法公开前发现风险。

9.5 Keras Lambda: 静态只能说可疑, 动态能看到 system

演讲者最喜欢的例子是 Keras Lambda。静态扫描器能看到“这里有一个 Keras Lambda 函数”, 也知道 Lambda 可能携带 Python 字节码; 但 Lambda 有合法用途, 很多模型用它表达自定义层或小型变换。因此静态工具很容易只能给出灰色结论: 它可能安全, 也可能不安全, 工具无法可靠分析嵌入的 Python 字节码。

动态扫描器处理这个例子的方式很硬: 直接运行并追踪。若这个 Lambda 只是做张量变换, 行为会留在模型正常边界内; 若它调用 `system`, 沙箱会记录命令执行。这里没有导入列表歧义, 也没有“Lambda 这个机制本身是否该被允许”的争论。真正被判定的是具体模型的具体运行行为。

²³源视频时间: 00:27:44; 来源区间: 00:27:40–00:27:50。

Issues 4			
SEVERITY ↓	ISSUE	DETECTED BY	SOURCE FILE
▲ Critical	aim-glibc-code-execution	⚡ Dynamic Scanner	model.keras ^
DETAILS glibc __libc_system called with arg0: echo "You've been pwned." ISSUE DESCRIPTION This model uses execution functions that are capable of running arbitrary commands on the host system during loading.			
REMEDIATION As no ML framework requires CLI commands as part of its loading or inference process, avoid using this model altogether.			
■ High	aim-glibc-process-manipulation	⚡ Dynamic Scanner	model.keras v
■ High	aim-sys-process-creation	⚡ Dynamic Scanner	model.keras v
■ Medium	aim-keras-unsafe-layer	⊕ Static Scanner	model.keras ^
DETAILS Use of unsafe Keras layer Lambda ISSUE DESCRIPTION This model uses Lambda layers that execute arbitrary code at both runtime and build time during model loading. Proper sanitization of this code requires Dynamic Scanning.			
REMEDIATION If this is a homegrown model, consider converting the Lambda layer to a standard, already-defined Keras layer if possible. If this is an untrusted model, use this model only if the Dynamic Scanner concluded there are no malicious actions embedded into the model.			

图 24: Keras Lambda 对比页把静态与动态的差别压在同一张截图里：同一个 `model.keras` 同时出现 Dynamic Scanner 的 Critical 命令执行、进程操纵、进程创建告警，以及 Static Scanner 的 Medium 级 `aim-keras-unsafe-layer`。静态扫描只能指出 Lambda 层可能危险；动态扫描进一步看到 `glibc__libc_system` 实际被调用，风险判断从“可疑结构”推进到“已观察行为”。²⁴

Lambda、字节码与行为证据

Keras Lambda 像一张可以夹带小纸条的模型图节点：合法开发者会在纸条上写自定义计算，攻击者也能写危险动作。静态扫描看到“有纸条”，却未必能完全读懂纸条在所有路径上的效果；动态扫描让纸条进入真实解释器，再观察它有没有触发模型不该触发的系统动作。

9.6 为什么动态方案正好回应静态短板

把第 3 到第 8 节的失败模式逐项对照，动态方案的优势会更清楚。

首先，拒绝列表失败是因为危险函数名无法穷举。动态扫描不用穷举所有包装函数，只需要定义模型不该产生的行为类别。其次，嵌入字节码把静态分析带入复杂性泥潭；动态扫描把具体样本交给 Python 和框架真实执行，观察结果，避开了“完备预测任意程序行为”的理论负担。再次，Pickle 状态不同步和 Joblib blob 都是“扫描器模拟路径”和“真实加载路径”分叉；动态执行走的就是加载器真实路径。最后，允许列表面对模糊原语和自定义架构会误伤业务；行为边界则可以允许 `getattr`、`auto_map`、自定义层和仓库代码存在，同时把它们是否越界交给运行时追踪来判断。

²⁴源视频时间：00:28:12；来源区间：00:28:10–00:29:35；Q&A 在 00:31:17–00:32:03 再次解释此图。

不要把动态扫描理解成万能沙箱

沙箱并非魔法。攻击者可能做环境探测、延迟触发、输入触发、资源耗尽或反沙箱规避。工程上仍需限制权限、阻断真实外联、模拟常见环境、设计多组输入、记录高质量事件，并把 `unknown` 作为需要管控的状态。动态扫描的价值在于把关键判断推进真实行为，不能替代完整的模型供应链治理。

9.7 静态与动态的反馈闭环

演讲者还强调了一个很实际的收益：动态发现可以反哺静态规则。以 `MLflow` 包装函数为例，动态扫描看到它导致进程创建后，安全团队就可以把这条路径补进静态签名或静态风险规则里。下一次遇到同类模型，静态扫描阶段就能更早提示风险，动态扫描则继续负责捕捉未知路径。

这形成一个闭环：静态扫描负责快速筛查已知风险和明显异常，动态沙箱负责验证未知路径的真实行为；动态沙箱发现新危险后，产出的函数名、调用链、系统调用特征和参数模式又能更新静态签名。两者的关系像门口安检和室内传感器：前者便宜快速，后者看得更深；把后者的发现拿回去训练前者，整个准入系统才会逐步变聪明。

本节的安全结论

模型动态扫描器的定义可以压缩成一句话：在隔离环境中加载并运行模型，记录 $O(D, M, p)$ ，只要观察行为与 A_{bad} 相交，就判定 $V(M) = \text{unsafe}$ 。这个规则直接对准前面所有静态绕过的共同漏洞：文件里的静态影子会骗人，运行时副作用更难伪装成正常模型计算。

9.8 本章小结

本章给出了前面绕过链条的正面解法：把模型加载和推理放进沙箱 D ，追踪观察行为 $O(D, M, p)$ ，并用异常行为集合 A_{bad} 做裁决。正常模型行为边界相对狭窄，主要是读取模型文件、加载权重、操作内存和张量、执行框架计算；命令行执行、进程创建、任意网络连接和数据发送都应被视为越界信号。演讲中的动态结果也印证了这一点：`Joblib` 案例暴露 `system` 调用，`MLflow` 包装函数暴露进程创建，网络案例暴露连接与发送数据，`Keras Lambda` 从静态灰区变成了明确的 `system` 行为证据。更重要的是，动态发现还能反哺静态签名，让快速静态筛查和深度行为追踪形成迭代闭环。

10 总结与问答：动态检测的边界、优势与实践落点

到 00:29:52 左右，演讲已经完成了从问题到方案的闭环：第三方模型会把外部代码、字节码、序列化对象和仓库脚本带进本地执行边界；静态扫描器试图在不运行模型的前提下判断风险；攻击者则利用包装函数、嵌入字节码、`Pickle` 状态不同步、`Joblib` 动态字节块、`getattr` 与自定义架构，把真实执行行为藏到扫描器看不清的路径里。结尾部分的价值在于把这些细节重新压缩成一个工程判断：模型安全检查的对象，最终应当落到加载和推理时的行为。

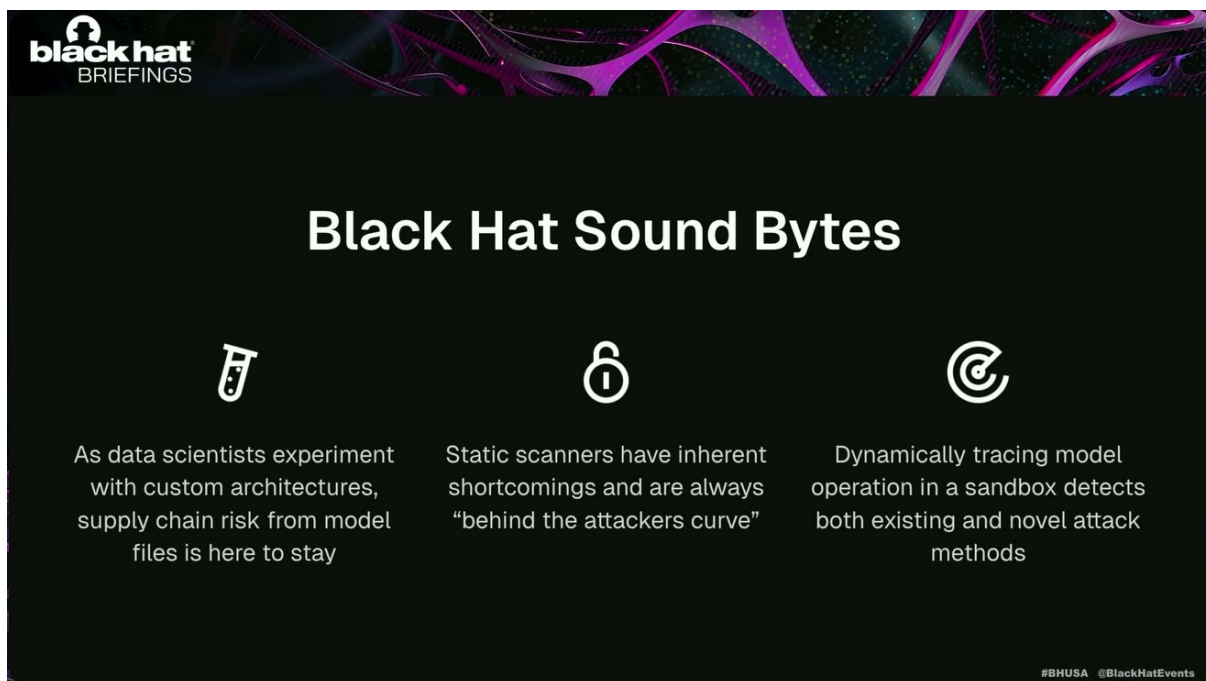


图 25: 收束页给出三条最终结论：数据科学家持续试验自定义架构，因此模型文件供应链风险会长期存在；静态扫描器有内在短板，并且会落在攻击曲线之后；在沙箱里动态追踪模型操作，可以同时发现既有攻击方法和新型攻击方法。这一页适合放在总结段开头，把第 9 节的技术证据转成第 10 节的实践结论。²⁵

10.1 演讲收束：为什么模型扫描仍然绕不开

演讲者的第一点回顾是现实层面的：数据科学家高度依赖第三方模型。Hugging Face、研究仓库、框架示例和企业内部 fork 已经成了日常研发材料。模型准入因此类似软件供应链准入，只是交付物从依赖包、镜像和脚本，扩展到了权重、配置、序列化结构、模型架构和仓库代码。一个团队把外部模型下载到笔记本、训练集群或推理服务里，等价于把外部制品带入组织的执行环境。

第二点回顾是格式层面的：SafeTensors 这类格式能降低权重文件承载任意对象反序列化的风险，但它无法消除自定义架构带来的加载路径。很多新模型先以自定义架构形式出现，等使用广泛后才可能被主流框架吸收。DeepSeek 这类架构、Kimi K2 这类新模型，都会让读者看到同一条规律：今天的自定义代码，可能是明天的主流组件；今天若完全封锁它，研发流程会受阻；今天若无条件信任它，执行风险会进入环境。

SafeTensors 与自定义架构的边界

SafeTensors 更像把权重 W 放进一个更克制的容器里，它处理的是张量存储层的一大类风险。完整模型仍然可以写成 $M = (W, A)$ ，其中 A 包含架构、加载逻辑、配置映射和仓库 Python 文件。安全扫描若只盯住权重格式，就像只检查机器零件的材质，却没有检查装配说明和自动装配脚本。

第三点回顾是安全方法论层面的：静态扫描器有内在短板，而且天然滞后于攻击者。拒绝列表要追逐无穷无尽的危险函数名称，允许列表又会被真实机器学习生态中的模糊原语和非标准依赖挤压。

²⁵ 源视频时间：00:29:55；来源区间：00:29:52–00:30:20。

静态分析还要面对程序语义预测问题：Keras Lambda、Dill、Cloudpickle、Pickle 虚拟机和 Joblib 动态块都在提醒读者，字节层面的“看见”不等于行为层面的“理解”。

结尾的中心判断

动态追踪的优势来自一个更小的判定空间：良性模型加载与推理通常只需要读取模型文件、加载权重、分配内存、执行张量计算和返回结果。进程创建、命令行执行、任意网络连接、可疑写文件、数据外传等行为，落在模型正常操作边界之外。安全系统观察这些行为，比枚举所有可通向 RCE 的函数名更可控。

可以把这个判断写成一个简化规则：

$$O(D, M, p) \cap A_{\text{bad}} \neq \emptyset \Rightarrow V(M) = \text{unsafe}$$

- D ：执行模型的沙箱与追踪环境。
- M ：待检查的模型制品，包含权重、架构和可能的仓库代码。
- p ：用于触发加载或推理路径的输入。
- $O(D, M, p)$ ：沙箱中实际观察到的操作集合或操作序列。
- A_{bad} ：异常行为集合，字幕明确点名的例子包括 shell 执行、子进程创建、异常网络流量和向外部地址发送数据；可疑文件写入等副作用可作为工程扩展继续纳入。
- $V(M)$ ：模型准入结论。

这条规则的关键含义很直接：它不要求扫描器提前知道攻击者使用了 MLflow 包装函数、Pickle opcode 技巧、Keras Lambda 字节码，还是某个新出现的库函数。只要最终行为越过模型正常操作边界，动态追踪就有机会把它记录下来。

10.2 问答一：截图里的工具到底在看什么

第一位听众追问 00:31:17–00:32:02 中展示的静态/动态扫描截图，核心问题是：那张图里使用的到底是什么工具或样本？演讲者澄清，这里展示的是一个带有 Keras Lambda 函数的模型。这个 Lambda 函数表面上只是自定义模型层的一段逻辑，实际会执行 CLI 命令。

00:31:17–00:32:02：Keras Lambda 截图澄清

听众：你展示静态和动态扫描器截图时，用的是什麼？

演讲者：那是一个带有 Keras Lambda 函数的模型。这个 Lambda 函数实际会调用并执行 CLI 命令。静态扫描器只能说这里有一个 Keras Lambda 函数，可能安全，也可能不安全；动态扫描器能看到它实际运行了 CLI 命令。

这段问答的教学价值在于，它把“静态不确定性”和“动态可见性”放在同一个例子里。Keras Lambda 是真实框架功能，用来表达自定义计算并不罕见；问题在于，静态扫描看到的是一个可能良性的自定义函数容器，动态追踪看到的是命令行执行这个具体副作用。前者给出“也许有风险”的模糊判断，后者把风险压到可审计的系统行为上。

这一问答继续围绕图 24 展开，因此正文只回指前图，避免重复插入同一张 Keras Lambda 对比页。

不要把 Lambda 本身当成唯一风险信号

Keras Lambda 的出现只能说明模型携带自定义计算逻辑。真正决定安全结论的，是这段逻辑在加载或推理时做了什么：普通张量变换属于模型语义，CLI 命令、子进程和网络连接则越过了模型正常行为边界。把“出现 Lambda”直接等同于恶意会误伤正常模型；把“只是 Lambda”当作安全也会漏掉命令执行。

10.3 问答二：动态检测会不会变成无限枚举 RCE

第二位听众提出了更尖锐的问题：动态方法会不会重新进入杀毒软件长期面对的困境，即不断寻找不同类型的 RCE 和执行技巧？这个问题很关键，因为如果动态扫描依旧要穷举所有执行手法，那它只是在另一个层面重复拒绝列表的老路。

演讲者的回答是整场演讲的最后一次概念校准：扫描器不需要提出所有 RCE 方法。它只需要知道模型应该执行一组非常具体、非常有限的操作；除此之外的行为，就应被视为“不正常”。听众随后把它概括为 allow listing approach，演讲者补充说，这里的允许列表之所以可行，是因为模型的已批准操作集合比静态方法里的函数、库、导入和 opcode 集合小得多。

00:32:12–00:33:02：RCE 枚举挑战

听众：这会不会变成不断找各种 RCE 和执行技巧？杀毒软件已经这样做很多年了。

演讲者：我们不需要想出所有 RCE 方法。我们知道模型应该只做一组很具体、很有限的操作，其他行为都可以认为不正常。这里的允许列表更容易建立，因为它是模型操作的批准集合。

这里要分清两种允许列表。静态允许列表列的是名字：允许哪些库、函数、导入、opcode、序列化原语。它会遇到 `getattr`、`ultralytics`、`auto_map` 和自定义架构这类生态现实。行为允许列表列的是动作：模型加载和推理时允许读哪些文件、使用哪些计算资源、是否允许联网、是否允许启动子进程、是否允许写出文件。后一种列表贴近“模型应该像模型一样工作”这个边界，因此更适合做最后一道准入判断。

行为允许列表的实质

动态检测的核心是维护模型正常行为边界，而非追逐所有坏函数名。这个边界可以按任务、框架和部署场景细化：离线图像分类模型通常不需要联网；权重加载不应启动 shell；推理服务不应在请求触发时写入陌生可执行文件。安全策略从“识别攻击技巧”转向“约束模型可做的事”。

10.4 实践落点：把动态扫描放进模型准入流程

对企业和研究团队来说，最后的落点并非把静态扫描丢掉。更稳的做法是把模型准入做成分层流程：先用静态扫描快速发现已知危险信号、格式异常、可疑导入和仓库脚本；再对外部模型、长尾模型、含自定义架构的模型、含 Lambda/bytecode/unsafe serialization 的模型执行动态沙箱追踪；最后把动态发现回灌给静态规则，使静态扫描能更快拦截重复样本和已知路径。

一个可执行的模型 intake 工作流可以这样组织：

1. 固定来源与版本：记录模型仓库、提交哈希、权重文件摘要、配置文件和所有随仓库下载的 Python 文件。
2. 做静态预筛：检查 Pickle、Joblib、Keras Lambda、`auto_map`、自定义 `modeling_*.py`、可疑导入和未知序列化结构。
3. 进入沙箱加载：在无生产凭证、无内部网络默认访问、文件系统权限受限的环境中加载模型。
4. 跑最小推理集：用代表性输入触发常见执行路径，记录进程、命令、网络、文件写入、动态库加载和异常系统调用。
5. 建立行为基线：把正常模型的读文件、内存、GPU、框架计算行为沉淀为基线，将越界行为转入人工复核或自动阻断。
6. 回灌规则：把动态确认的恶意路径转成静态签名、YARA/规则、仓库策略或框架加载限制。

动态扫描也有边界

动态方法只能观察被触发的路径。若恶意逻辑依赖特定输入、日期、环境变量、硬件特征或外部服务响应，单次沙箱运行可能看不到完整行为。因此实践上需要多输入、多环境、权限最小化、网络仿真、超时控制和人工复核队列。动态追踪显著提高可见性，但它不等于一次运行后永久安心。

10.5 仍然开放的问题与下一步

本讲给出的方向很清楚，但工程化落地还留下几类开放问题。第一，正常行为边界要按场景细分：训练、微调、离线推理、在线推理、RAG 检索和多模态预处理需要的文件、网络 and 进程权限不同。第二，沙箱成本需要控制：大模型加载消耗 GPU、内存和时间，企业需要决定哪些模型必须深度动态扫描，哪些模型可以走轻量流程。第三，触发覆盖率需要提高：只加载模型可能漏掉推理期后门，只跑一个输入可能漏掉条件触发路径。第四，仓库代码扫描要纳入同一条链路：权重文件、配置、tokenizer、custom layer、`auto_map` 指向的 Python 文件都属于同一个模型制品。

读者容易漏掉的反向概念：模型能力边界

安全上的“正常行为边界”可以和系统设计里的“能力边界”对应起来。能力边界回答一个对象被允许做什么：读哪些路径、访问哪些网络、调用哪些系统接口、获得哪些凭证。模型 intake 若只问“这个文件有没有已知恶意特征”，就会漏掉能力控制；若进一步问“这个模型即使恶意，最多能做什么”，就进入了更成熟的供应链防御思路。

因此，最务实的下一步是把模型当作可执行供应链制品管理：固定版本、记录来源、扫描全仓库、隔离加载、追踪行为、限制权限、保留审计证据。对安全团队来说，动态沙箱输出应进入告警和规则体系；对数据科学团队来说，模型下载与试验应有默认隔离环境；对平台团队来说，模型部署流水线应把“未知模型直接进入生产推理服务”视为高风险例外。

10.6 本章小结

本节把整场演讲压缩成三个结论。第一，第三方模型依赖已经成为 AI 研发常态，模型文件和模型仓库都应进入供应链安全视野。第二，SafeTensors 能改善权重存储风险，但自定义架构、Keras

Lambda、`auto_map` 和仓库 Python 文件使模型扫描继续必要。第三，静态扫描在函数名、导入、opcode 和格式解析层面会持续落后；动态追踪把判断对象转向运行时行为，尤其适合识别命令行执行、子进程创建、网络连接和数据发送这些字幕明确点名的异常操作。

问答部分补上了两个关键校准：Keras Lambda 示例说明，静态扫描可能只能指出“这里有一个模糊自定义函数”，动态扫描能看到它实际执行 CLI 命令；RCE 枚举质疑说明，动态检测的核心是模型操作的行为允许列表，而非穷举所有攻击技巧。最终的实践落点很硬：外部模型、长尾模型、自定义架构模型和含可执行加载逻辑的模型，在进入数据科学工作流和生产服务前，都需要比“看起来是 SafeTensors”更深的准入检查。